

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

ОСНОВИ ІНФОРМАТИКИ ТА ПРОГРАМУВАННЯ

Практикум

*Рекомендовано Методичною радою КПІ ім. Ігоря Сікорського
як навчальний посібник для здобувачів першого (бакалаврського) рівня вищої
освіти за освітньою програмою «Комп'ютерні технології в біології та
медицині» спеціальності 122 «Комп'ютерні науки»*

Укладачі: І. В. Федорін

Електронне мережеве навчальне видання

Київ
КПІ ім. Ігоря Сікорського
2023

УДК 004.021 (045)
Ф31

Укладачі: *Федорін Ілля Валерійович*, канд. фіз.-мат. наук, доцент кафедри біомедичної кібернетики, НТУУ КПІ імені Ігоря Сікорського

Рецензент: *Олійник А.С.*, д.фіз.-мат.н., доцент, професор кафедри алгебри і комп'ютерної математики Київського національного університету імені Тараса Шевченка

Бесараб О.Б., к.т.н., доцент, в.о. завідувача кафедри трансляційної медичної біоінженерії, Факультет біомедичної інженерії, НТУУ КПІ імені Ігоря Сікорського

Відповідальний
редактор

Городецька О. К., к.т.н., доцент кафедри біомедичної кібернетики, НТУУ КПІ імені Ігоря Сікорського

*Гриф надано Методичною радою КПІ ім. Ігоря Сікорського
(протокол № 3 від 07.12.2023 р.)
за поданням вченої ради факультету біомедичної інженерії
(протокол № 2 від 30.10.2023 р.)*

Ф31

Основи інформатики та програмування. [Електронний ресурс] : практикум : навч. посіб. для здобувачів ступеня бакалавра за освіт. програмою «Комп'ютерні технології в біології та медицині» спец. 122 «Комп'ютерні науки» / КПІ ім. Ігоря Сікорського; уклад.: І. В. Федорін; – Електрон. текст. дані (1 файл). – Київ: КПІ ім. Ігоря Сікорського, 2023. – 89 с.

Навчальний посібник призначено для закріплення на практичних заняттях теоретичних знань з основ інформатики та програмування. Посібник охоплює важливі концепції програмування мовою Python, включаючи роботу з типами даних, файлами та модулями. Усі завдання містять різноманітні сценарії і випадки використання, що спонукають студентів розуміти і застосовувати принципи програмування в різних контекстах.

УДК 004.021 (045)

Реєстр. № НП 23/24-177. Обсяг 3,7 авт. арк.

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
проспект Берестейський, 37, м. Київ, 03056
<https://kpi.ua>

Свідectво про внесення до Державного реєстру видавців, виготовлювачів і розповсюджувачів видавничої продукції ДК № 5354 від 25.05.2017 р.

© КПІ ім. Ігоря Сікорського, 2023

ЗМІСТ

Вступ.....	4
Практична робота №1: Типи даних у Python	5
Практична робота №2: Умовні оператори та логічні вирази	14
Практична робота №3: Цикли у Python	21
Практична робота №4: Системи числення	27
Практична робота №5: Списки у Python.....	34
Практична робота №6: Рядки у Python	40
Практична робота №7: Словники та множини у Python	47
Практична робота №8: Файли та методи рядків у Python	54
Практична робота №9: Створення графічного інтерфейсу у Python	65
Практична робота №10: ООП. Класи у Python	73
Практична робота №11: Регулярні вирази у Python	81
Список використаних джерел	89

ВСТУП

Практичний курс з інформатики та основ програмування зосереджений на набутті студентами практичних навичок і вмінь, які необхідні для ефективної роботи з алгоритмами, програмуванням, комп'ютерними системами та обробкою даних.

Дані комп'ютерні практикуми включають в себе ряд вправ, які дозволяють студентам використовувати та розуміти основні принципи програмування на Python, включаючи роботу з основними типами даних, операціями, умовними операторами, циклами, функціями, обробкою винятків та роботою з модулями.

Крім того, студенти зможуть навчитися роботі з колекціями даних в Python, включаючи списки, кортежі, словники та множини, а також навчатися працювати з файлами і використовувати регулярні вирази для пошуку та аналізу даних. Курс також знайомить студентів з використанням бібліотек Python для наукових обчислень та основами створення графічного інтерфейсу користувача.

Кожна тема курсу супроводжується відповідним набором вправ, які дозволяють студентам закріпити матеріал, навчитися розв'язувати проблеми та вдосконалити навички програмування. Завдання практикуму розроблені таким чином, щоб кожне нове завдання будувалося на основі попереднього, формуючи тим самим послідовний і логічний підхід до навчання.

По завершенні цього практикуму, студенти отримають не тільки теоретичні знання, але й практичні навички, які можна використовувати в реальному світі. Ці навички стануть цінним доповненням до їхнього резюме та портфолію, а також допоможуть студентам зробити перший крок на шляху до кар'єри в галузі інформаційних технологій.

ПРАКТИЧНА РОБОТА №1: ТИПИ ДАНИХ У PYTHON

Мета роботи – засвоїти типи даних у Python та особливості роботи з ними.

Теоретичні відомості

Типи даних у Python.

Python підтримує різноманітні типи даних, які дозволяють зберігати і маніпулювати різними видами інформації. Основні типи даних в Python включають:

1. Цілі числа (integers):

Цілі числа – це числові дані без десяткової частини. Вони можуть бути позитивними або негативними. Наприклад: 5, -10, 0. У Python цілі числа мають тип `int`.

2. Дійсні числа (floats):

Дійсні числа – це числові дані з десятковою частиною. Вони представляють числа з плаваючою комою. Наприклад: 3.14, 2.0, -0.5. У Python дійсні числа мають тип `float`.

3. Строки (strings):

Строки – це послідовності символів, які огортаються одинарними або подвійними лапками. Вони використовуються для зберігання текстової інформації. Наприклад: "Hello", 'Python'. У Python стрічки мають тип `str`.

4. Булеві значення (booleans):

Булеві значення – це тип даних, який може мати лише два можливі значення: `True` (істина) або `False` (хибність). Вони використовуються для представлення логічних значень і використовуються в умовних виразах. У Python булеві значення мають тип `bool`.

Ці типи даних є вбудованими типами в Python і мають різні операції та методи для маніпуляції з ними. Крім цих основних типів, Python також має багато інших вбудованих типів даних, таких як списки, кортежі,

словники, множини тощо. Кожен з цих типів даних має свої особливості і може бути використаний для різних завдань у програмуванні.

Розрахунок формул в Python.

В Python для виконання математичних операцій використовуються арифметичні оператори. Наприклад, `*` для множення, `-` для віднімання, `//` для цілочисельного ділення, `**` для піднесення до ступеня, `/` для ділення.

Приклад:

```
result = (7 * 6 - (18 // 3) ** 2) / 2.9  
print(result, type(result))
```

У цьому прикладі ми використовуємо оператори `*`, `-`, `//`, `**` та `/` для обчислення виразу. Функція `print()` виводить результат та його тип.

Робота зі строками в Python.

В Python строки можна створити, використовуючи одинарні або подвійні лапки. Ми можемо використовувати оператор `*` для повторення строки та оператор `+` для з'єднання строк.

Приклад:

```
string = "NLP" * 4 + "^"  
print(string, type(string))
```

У цьому прикладі ми використовуємо оператор `*` для створення 4 копій строки "NLP", а потім додаємо символ "^" до результату за допомогою оператора `+`. Функція `print()` виводить результат та його тип.

Обчислення виразу в Python.

Ви можете використовувати арифметичні оператори для обчислення математичних виразів в Python.

Приклад:

```
result = 5 * 5 - 7 // 2 ** 3 / 4  
print(result, type(result))
```

У цьому прикладі ми використовуємо оператори `*`, `-`, `//`, `**` та `/` для обчислення виразу. Функція `print()` виводить результат та його тип.

Версії Python та необхідні бібліотеки.

Наразі підтримуються та використовуються дві версії Python 2.7 та 3.9. Python 3.0 ввів у мову багато змін, тому код, написаний для версії 2.7, може не працювати під 3.9 і навпаки. Ми будемо використовувати Python версії 3.5 та старше (<https://www.python.org/downloads/>). Ви можете перевірити версію Python, що встановлена на вашому комп'ютері запустивши із командної строки `python --version`. Також для наступної роботи буде доречно встановити програму PyCharm community (безкоштовна програма) разом із бібліотекою Anaconda (використовуйте офіційні сайти програм для установки, вибравши версію, що відповідає вашій операційній системі та характеристикам комп'ютера <https://www.jetbrains.com/ru-ru/pycharm/download/#section=windows>, <https://www.anaconda.com/distribution/>).

Практична частина

Варіант 1:

1. Виконайте розрахунок формули: $((25 + 18) ** 2) / (2.2 + 7.3) - (12 \% 5)$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 7 копій строки "123", розділених комою. Виведіть отриманий результат та тип даних.
3. Розрахуйте результат: $((123 + 57) // 5) ** 2 - (14.5 + 3.2 * 2)$. Виведіть результат та тип даних.

Варіант 2:

1. Виконайте розрахунок формули: $((39.5 + 12.3) ** 3) / (18.5 - 4.2) + (20 \% 3)$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 5 копій строки "Python", розділених символом "_". Виведіть отриманий результат та тип даних.

3. Розрахуйте результат: $((89 + 35) // 7) ** 3 - (9.5 + 5.6 * 3)$. Виведіть результат та тип даних.

Варіант 3:

1. Виконайте розрахунок формули: $((13 + 15) ** 2) / (3.4 + 5.6) - (7 \% 4)$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 4 копій строки "1234", розділених комою. Виведіть отриманий результат та тип даних.
3. Розрахуйте результат: $((56 + 74) // 3) ** 2 - (10.7 + 3.5 * 2)$. Виведіть результат та тип даних.

Варіант 4:

1. Виконайте розрахунок формули: $((25.7 + 15.3) ** 2) / (12.5 - 7.8) + (15 \% 7)$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 6 копій строки "C++", розділених символом "_". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат: $((78 + 56) // 7) ** 3 - (6.5 + 3.2 * 2)$. Виведіть результат та тип даних.

Варіант 5:

1. Виконайте розрахунок формули: $((40 + 30) ** 2) / (7.1 + 9.8) - (10 \% 3)$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 8 копій строки "4567", розділених комою. Виведіть отриманий результат та тип даних.
3. Розрахуйте результат: $((20 + 30) // 5) ** 3 - (15.2 + 2.8 * 3)$. Виведіть результат та тип даних.

Варіант 6:

1. Виконайте розрахунок формули: $((15.2 + 20.5) ** 3) / (10.3 - 5.6) + (12 \% 4)$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 7 копій строки "Java", розділених символом "_". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат: $((50 + 30) // 4) ** 2 - (12.3 + 4.6 * 2)$

Варіант 7:

1. Виконайте розрахунок формули: $(23 * 4 - (30 // 5) ** 3) / 2.7$. Виведіть отриманий результат та тип даних.
2. Створіть строку, що складається з 3-х копій рядка "123", розділених символом "#". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат: $(7 * 8 - 4 // 2 ** 3) / 5$. Виведіть результат та тип даних.

Варіант 8:

1. Виконайте розрахунок формули: $(4 ** 3 * 2 - (10 \% 3)) / 4.7$. Виведіть отриманий результат та тип даних.
2. Створіть строку, що складається з 5-ти копій рядка "Python", розділених символом "!". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат: $(9 * 3 - 8 // 2 ** 2) / 3.5$. Виведіть результат та тип даних.

Варіант 9:

1. Виконайте розрахунок формули: $(2 ** 4 * 3 - (15 // 2)) / 6.3$. Виведіть отриманий результат та тип даних.
2. Створіть строку, що складається з 6-ти копій рядка "Google", розділених символом "_". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат: $(6 * 2 - 5 // 3 ** 2) / 4$. Виведіть результат та тип даних.

Варіант 10:

1. Виконайте розрахунок формули: $(3 ** 3 * 4 - (8 \% 3)) / 7.9$. Виведіть отриманий результат та тип даних.
2. Створіть строку, що складається з 4-х копій рядка "1234", розділених символом " ". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат: $(6 * 12 - 5 // 23 ** 2) / 14$. Виведіть результат та тип даних.

Варіант 11:

1. Виконайте розрахунок формули: $(10 ** 3 * 2 - (20 // 5)) / 4.8$. Виведіть отриманий результат та тип даних.

2. Створіть строку, що складається з 2-х копій рядка "KLS", розділених символом "-". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат: $(6 * 7 - 5 // 2 ** 4) / 3$. Виведіть результат та тип даних.

Варіант 12:

1. Виконайте розрахунок формули: $(3 ** 2 * 4 - (12 \% 3)) / 5.7$. Виведіть отриманий результат та тип даних.
2. Створіть строку, що складається з 3-х копій рядка "AI", розділених символом "!". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат: $(8 * 3 - 7 // 2 ** 3) / 4.5$. Виведіть результат та тип даних.

Варіант 13:

1. Виконайте розрахунок формули: $(4 ** 3 * 2 - (15 // 2)) / 6.9$. Виведіть отриманий результат та тип даних.
2. Створіть строку, що складається з 6-ти копій рядка "Google", розділених символом "/". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат: $(5 * 2 - 6 // 4 ** 2) / 5.5$. Виведіть результат та тип даних.

Варіант 14:

1. Виконайте розрахунок формули: $(3 * 2 - (18 // 3) ** 2) / 2.5$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 5 копій строки "coding", розділених символом "_". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат $8 * 7 - 9 // 2 ** 2 / 3$. Виведіть результат та тип даних.

Варіант 15:

1. Виконайте розрахунок формули: $(10 ** 2 * 3 - (7 \% 2)) / 4.1$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 3 копій строки "Pythonic", розділених символом "-". Виведіть отриманий результат та тип даних.

3. Розрахуйте результат $12 * 7 - 4 // 3 ** 2 / 2$. Виведіть результат та тип даних.

Варіант 16:

1. Виконайте розрахунок формули: $(6 * 3 - (9 // 3) ** 2) / 3$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 4 копій строки "AI", розділених символом "|". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат $5 * 6 - 7 // 2 ** 3 / 4$. Виведіть результат та тип даних.

Варіант 17:

1. Виконайте розрахунок формули: $(8 ** 2 * 3 - (4 \% 2)) / 5$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 2 копій строки "Data", розділених символом "#". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат $13 * 6 - 3 // 2 ** 2 / 5$. Виведіть результат та тип даних.

Варіант 18:

1. Виконайте розрахунок формули: $(2 * 4 - (16 // 4) ** 3) / 2$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 6 копій строки "Machine", розділених символом "+". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат $8 * 9 - 10 // 3 ** 2 / 6$. Виведіть результат та тип даних.

Варіант 19:

1. Виконайте розрахунок формули: $(11 ** 3 * 2 - (5 \% 3)) / 4$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 3 копій строки "Learning", розділених символом "-". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат $12 * 8 - 5 // 3 ** 3 / 2$. Виведіть результат та тип даних.

Варіант 20:

1. Виконайте розрахунок формули: $(5 * 3 - (9 // 3) ** 2) / 3.6$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 4 копій строки "ML", розділених символом "*". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат $6 * 6 - 8 // 2 ** 4 / 4$. Виведіть результат та тип даних.

Варіант 21:

1. Виконайте розрахунок формули: $(9 ** 3 * 3 - (4 \% 3)) / 5.5$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 2 копій строки "DL", розділених символом "#". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат $14 * 7 - 4 // 3 ** 3 / 5$. Виведіть результат та тип даних.

Варіант 22:

1. Виконайте розрахунок формули: $(4 * 7 - (13 \% 3) ** 2) / 3.5$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 3 копій строки "Data", розділених символом "=". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат $7 * 6 - 10 // 2 ** 4 / 2$. Виведіть результат та тип даних.

Варіант 23:

1. Виконайте розрахунок формули: $(5 ** 2 * 4 - (2 \% 2)) / 4.3$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 5 копій строки "AI", розділених символом "/". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат $16 * 9 - 4 // 3 ** 2 / 3$. Виведіть результат та тип даних.

Варіант 24:

1. Виконайте розрахунок формули: $(7 * 6 - (18 // 3) ** 2) / 2.9$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 4 копій строки "NLP", розділених символом "^". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат $5 * 5 - 7 // 2 ** 3 / 4$. Виведіть результат та тип даних.

Варіант 25:

1. Виконайте розрахунок формули: $(6 ** 2 * 5 - (3 \% 2)) / 3.8$. Виведіть отриманий результат та тип даних.
2. Створіть строку, яка складається з 2 копій строки "CV", розділених символом "&". Виведіть отриманий результат та тип даних.
3. Розрахуйте результат $13 * 8 - 6 // 2 ** 2 / 5$. Виведіть результат та тип даних.

Контрольні запитання

1. Що таке тип даних в програмуванні?
2. Які основні типи даних є в Python?
3. Які операції можна виконувати зі стрічками в Python?
4. Що таке булеві значення і як вони використовуються в програмуванні?
5. Які операції можна виконувати з числами в Python?

ПРАКТИЧНА РОБОТА №2: УМОВНІ ОПЕРАТОРИ ТА ЛОГІЧНІ ВИРАЗИ

Мета роботи – Ознайомитися з логічним типом даних у Python, умовними операторами та виконанням умовних інструкцій. Зрозуміти, як формувати логічні вирази та використовувати їх для контролю потоку в програмах на Python. Навчитися розв'язувати практичні завдання за допомогою умовних операторів.

Теоретичні відомості

Логічний тип даних. В Python логічний тип даних представлений лише двома значеннями: True (істина) та False (неправда). Ці значення використовуються для репрезентації істинності або хибності виразу.

```
print(type(True)) # <class 'bool'>
print(type(False)) # <class 'bool'>
```

Логічні оператори. Python включає в себе три логічні оператори: and, or, not.

- and: Повертає True, якщо обидва вирази істинні.
- or: Повертає True, якщо хоча б один з виразів істинний.
- not: Повертає протилежне булеве значення від виразу.

```
x = True
y = False
print(x and y) # False
print(x or y) # True
print(not x) # False
print(not y) # True
```

Умовні оператори. Python використовує умовні оператори if, elif, else для виконання певних блоків коду на основі визначених умов.

- `if`: виконує блок коду, якщо умова істинна.
- `elif`: використовується для перевірки додаткових умов, якщо умови в попередніх `if` або `elif` були невірними.
- `else`: виконує блок коду, якщо усі умови в попередніх `if` та `elif` були невірними.

```
x = 10
if x > 0:
    print("x is positive")
elif x < 0:
    print("x is negative")
else:
    print("x is zero")
```

У вищенаведеному прикладі, перший вираз $x > 0$ істинний, тому блок коду всередині цього `if` виконується, і виводиться "x is positive".

Це лише основи логічних виразів та умовних операторів в Python. Реальні програми на Python будуть використовувати значно більш складні та розгалуджені логічні вирази та умовні конструкції.

Практична частина

Зауваження: у завданнях ввід даних відбувається користувачем, за допомогою функції вводу `input()`!

Варіант 1:

1. Дано три числа a , b і c . Якщо a найбільше з цих чисел, то обчислити abc . Якщо b найбільше, то обчислити $a+b+c$. Якщо c найбільше, то обчислити $a-b+c$.
2. Обчислити значення функції: для $x > 7$, $f(x) = x^3 + 7$; для $x < 7$, $f(x) = 7 * x - 2$; для $x = 7$, $f(x) = 50$.

Варіант 2:

1. Вводиться номер місяця. Вивести пору року, до якої він відноситься.

2. Обчислити значення функції: для $x > 8$, $f(x) = 2 \cdot x^2 + 8$; для $x < 8$, $f(x) = x + 8$; для $x = 8$, $f(x) = 64$.

Варіант 3:

1. Дано координати точки на площині (x, y) . Визначити, в якому координатному квадранті вона знаходиться.
2. Обчислити значення функції: для $x > 1$, $f(x) = x^3 - 1$; для $x < 1$, $f(x) = x - 1$; для $x = 1$, $f(x) = 0$.

Варіант 4:

1. Дано три числа a , b і c . Якщо a найбільше з цих чисел, то обчислити abc . Якщо b найбільше, то обчислити $a+b+c$. Якщо c найбільше, то обчислити $a-b+c$.
2. Обчислити значення функції: для $x > 3$, $f(x) = 2x^2 + 3$; для $x < 3$, $f(x) = 3x - 2$; для $x = 3$, $f(x) = 18$.

Варіант 5:

1. Дано три числа. Вивести їх в порядку зростання.
2. Обчислити значення функції: для $x > 0$, $f(x) = x^3 + 0$; для $x < 0$, $f(x) = 0 \cdot x - 5$; для $x = 0$, $f(x) = 5$.

Варіант 6:

1. Студент відвідує заняття, якщо температура повітря є від 10 до 27 градусів. Визначте, чи піде студент на заняття при заданій користувачем температурі повітря.
2. Обчислити значення функції: для $x > 2$, $f(x) = 3 \cdot x^2 + 2$; для $x < 2$, $f(x) = x + 2$; для $x = 2$, $f(x) = 14$.

Варіант 7:

1. Дано два числа. Якщо вони однакові, то присвоїти цим змінним нулі, інакше - значення, рівне більшому числу.
2. Обчислити значення функції: для $x > 4$, $f(x) = x^3 + 4$; для $x < 4$, $f(x) = 4 \cdot x - 1$; для $x = 4$, $f(x) = 60$.

Варіант 8:

1. Дано число. Якщо воно більше 3, але менше 7, то помножити його на 2. Якщо воно більше 7 і менше 40, то відняти 100. Якщо воно не підпадає під попередні варіанти, то занулити.
2. Обчислити значення функції: для $x > 5$, $f(x) = 2x^2 - 5$; для $x < 5$, $f(x) = 5x + 1$; для $x = 5$, $f(x) = 20$.

Варіант 9:

1. Дано координати трьох вершин трикутника (х,у для трьох точок). Визначити, чи є цей трикутник рівностороннім.
2. Обчислити значення функції: для $x > 6$, $f(x) = 3x^2 + 6$; для $x < 6$, $f(x) = 6x - 2$; для $x = 6$, $f(x) = 108$.

Варіант 10:

1. Вводиться буква. Визначити, чи є це голосна буква українського алфавіту.
2. Обчислити значення функції: для $x > 7$, $f(x) = 2 * x^2 + 7$; для $x < 7$, $f(x) = x + 7$; для $x = 7$, $f(x) = 105$.

Варіант 11:

1. Напишіть програму, що визначає, чи є введене число ступенем двійки.
2. Обчислити значення функції: для $x > 9$, $f(x) = x^3 + 9$; для $x < 9$, $f(x) = 9 * x - 1$; для $x = 9$, $f(x) = 720$.

Варіант 12:

1. Дано число. Визначте, чи є воно додатним, від'ємним, або нулем.
2. Обчислити значення функції: для $x > 10$, $f(x) = 2 * x^2 + 10$; для $x < 10$, $f(x) = x + 10$; для $x = 10$, $f(x) = 200$.

Варіант 13:

1. Введіть дві букви. Визначте, чи вони обидві великі або обидві малі.
2. Обчислити значення функції: для $x > 11$, $f(x) = x^3 - 11$; для $x < 11$, $f(x) = 11 * x - 1$; для $x = 11$, $f(x) = 1320$.

Варіант 14:

1. Визначте, чи є вказаний користувачем рік високосним.

2. Обчислити значення функції: для $x > 3$, $f(x) = 3x^2 + 3$; для $x < 3$, $f(x) = 3x - 1$; для $x = 3$, $f(x) = 30$.

Варіант 15:

1. Визначте, чи можна побудувати прямокутник із заданою довжиною та шириною на прямокутному полі заданого розміру.
2. Обчислити значення функції: для $x > 4$, $f(x) = x^3 + 4$; для $x < 4$, $f(x) = 4 * x - 2$; для $x = 4$, $f(x) = 60$.

Варіант 16:

1. Вводиться три числа. Виведіть найбільше з них.
2. Обчислити значення функції: для $x > 5$, $f(x) = 2 * x^2 + 5$; для $x < 5$, $f(x) = x + 5$; для $x = 5$, $f(x) = 45$.

Варіант 17:

1. Дано від'ємні числа a , b і c . Знайти найбільше з трьох чисел і обчислити його куб
2. Обчислити значення функції: для $x > 6$, $f(x) = x^3 + 6$; для $x < 6$, $f(x) = 6 * x - 3$; для $x = 6$, $f(x) = 222$.

Варіант 18:

1. Введіть три числа. Виведіть їх в порядку зростання.
2. Обчислити значення функції: для $x > 7$, $f(x) = x^2 + 7$; для $x < 7$, $f(x) = 7 * x - 1$; для $x = 7$, $f(x) = 56$.

Варіант 19:

1. Введіть три числа. Визначте, чи можуть вони бути довжинами сторін трикутника (Числа можуть бути довжинами сторін трикутника, якщо кожне з чисел менше суми двох інших)
2. Обчислити значення функції: для $x > 8$, $f(x) = 2 * x^2 + 8$; для $x < 8$, $f(x) = x + 8$; для $x = 8$, $f(x) = 128$.

Варіант 20:

1. Введіть число. Виведіть його зворотне значення ($1/n$) якщо воно від'ємне, та квадрат числа, якщо воно додатне.

2. Обчислити значення функції: для $x > 9$, $f(x) = x^3 + 9$; для $x < 9$, $f(x) = 9 * x - 1$; для $x = 9$, $f(x) = 720$.

Варіант 21:

1. Введіть число. Визначте, чи є воно парним.
2. Обчислити значення функції: для $x > 10$, $f(x) = x^2 + 10$; для $x < 10$, $f(x) = x + 10$; для $x = 10$, $f(x) = 200$.

Варіант 22:

1. Введіть число. Виведіть, чи є воно додатним, від'ємним, або нулем.
2. Обчислити значення функції: для $x > 11$, $f(x) = x^3 + 11$; для $x < 11$, $f(x) = 11 * x - 1$; для $x = 11$, $f(x) = 1331$.

Варіант 23:

1. Дано два числа. Визначте, чи є хоча б одне з них дійсним, і якщо так, виведіть це число.
2. Обчислити значення функції: для $x > 12$, $f(x) = 2 * x^2 + 12$; для $x < 12$, $f(x) = x + 12$; для $x = 12$, $f(x) = 288$.

Варіант 24:

1. Введіть число. Виведіть, чи є воно простим.
2. Обчислити значення функції: для $x > 13$, $f(x) = x^3 + 13$; для $x < 13$, $f(x) = 13 * x - 1$; для $x = 13$, $f(x) = 2197$.

Варіант 25:

1. Для заданого року визначити значення століття (наприклад, 1900 рік - 19 століття, 1901 рік - 20 століття).
2. Обчислити значення функції: для $x > 14$, $f(x) = 2 * x^2 + 14$; для $x < 14$, $f(x) = x + 14$; для $x = 14$, $f(x) = 392$.

Контрольні запитання

1. Що таке логічний тип даних в Python?
2. Які оператори використовуються для формування логічних виразів у Python?

3. Як в Python використовуються умовні оператори? Поясніть, як вони контролюють потік програми.
4. Яка різниця між операторами if, elif та else?
5. Як можна використати умовні оператори для розв'язання практичних задач?
6. Як працює оператор not та як його можна використати при формуванні логічних виразів?
7. Що таке вкладені умовні оператори та коли їх використовують?
8. Як розуміти пріоритет операторів у логічних виразах?
9. Що відбувається, якщо в умовному виразі використовуються типи даних, що не є булевими?
10. Дайте приклад складного логічного виразу та поясніть, як він обчислюється

ПРАКТИЧНА РОБОТА №3: ЦИКЛИ У PYTHON

Мета роботи – засвоїти принципи роботи циклів в Python, а також практично застосувати ці знання для розв'язання задач.

Теоретичні відомості

Цикли є основною концепцією в програмуванні, що дозволяє виконувати однаковий набір інструкцій декілька разів. В Python є два основні типи циклів: `for` та `while`.

Цикл `for`: Цей цикл використовується для ітерації через послідовність (список, кортеж, рядок) або інший об'єкт, який можна ітерувати. Синтаксис циклу `for`:

```
for val in sequence:  
    # виконуємо дії
```

Наприклад, якщо ми маємо список чисел та хочемо вивести кожне число, ми можемо зробити це наступним чином:

```
numbers = [1, 2, 3, 4, 5]  
for num in numbers:  
    print(num)
```

Цикл `while`: Цикл `while` виконує інструкції, допоки умова є істинною. Синтаксис циклу `while`:

```
while condition:  
    # виконуємо дії
```

Наприклад, якщо ми хочемо вивести числа від 1 до 5, ми можемо зробити це за допомогою циклу `while`:

```
num = 1  
while num <= 5:
```

```
print(num)
```

```
num += 1
```

Оператори break та continue: Оператор break використовується для виходу з циклу негайно, обхідно обмеження умови циклу. Оператор continue переходить до наступної ітерації, пропускаючи решту коду в поточному циклі.

Вкладені цикли: В Python можливо використовувати цикл в середині іншого циклу, це називається вкладеним циклом. Це особливо корисно при роботі з даними, які мають багатовимірну структуру.

```
for i in range(3):  
    for j in range(3):  
        print(i, j)
```

Цей код виведе всі пари індексів для 3x3 матриці.

Практична частина

Варіант 1:

1. Обчисліть значення функції $y = \sin(x)$ на відрізку $[0; \pi]$ з кроком 0.1 за допомогою циклу while.
2. Створіть програму, яка визначає індекс найменшого елемента послідовності натуральних чисел, що завершується числом 0.

Варіант 2:

1. Обчисліть значення функції $y = \cos(x)$ на відрізку $[0; \pi/2]$ з кроком 0.1 за допомогою циклу for.
2. Визначте суму непарних елементів послідовності натуральних чисел, що завершується числом 0.

Варіант 3:

1. Обчисліть значення функції $y = \tan(x)$ на відрізку $[0; \pi/4]$ з кроком 0.1 за допомогою циклу while.

2. Визначте кількість додатних елементів послідовності цілих чисел, що завершується числом 0.

Варіант 4:

1. Обчисліть значення функції $y = \ln(x)$ на відрізку $[1; e]$ з кроком 0.1 за допомогою циклу `for`.
2. Знайдіть перший від'ємний елемент послідовності дійсних чисел, що завершується числом 0.

Варіант 5:

1. Обчисліть значення функції $y = \sqrt{x}$ на відрізку $[0; 10]$ з кроком 0.1 за допомогою циклу `while`.
2. Знайдіть суму квадратів перших n натуральних чисел.

Варіант 6:

1. Обчисліть значення функції $y = 1/x$ на відрізку $[1; 10]$ з кроком 0.1 за допомогою циклу `for`.
2. Знайдіть суму кубів перших n натуральних чисел.

Варіант 7:

1. Обчисліть значення функції $y = e^x$ на відрізку $[0; 1]$ з кроком 0.1 за допомогою циклу `while`.
2. Знайдіть добуток непарних елементів послідовності натуральних чисел, що завершується числом 0.

Варіант 8:

1. Обчисліть значення функції $y = x^3$ на відрізку $[0; 5]$ з кроком 0.1 за допомогою циклу `for`.
2. Визначте кількість елементів послідовності цілих чисел, що є кратними числу 3 і завершується числом 0.

Варіант 9:

1. Обчисліть значення функції $y = 3^x$ на відрізку $[0; 3]$ з кроком 0.1 за допомогою циклу `while`.
2. Знайдіть середнє арифметичне елементів послідовності дійсних чисел, що завершується числом 0.

Варіант 10:

1. Обчисліть значення функції $y = \log(x)$ на відрізку $[1; 10]$ з кроком 0.1 за допомогою циклу `for`.
2. Визначте найменший непарний елемент послідовності натуральних чисел, що завершується числом 0.

Варіант 11:

1. Обчисліть значення функції $y = 2^x$ на відрізку $[0; 5]$ з кроком 0.1 за допомогою циклу `while`.
2. Визначте найбільший парний елемент послідовності натуральних чисел, що завершується числом 0.

Варіант 12:

1. Обчисліть значення функції $y = x^2$ на відрізку $[-5; 5]$ з кроком 0.1 за допомогою циклу `for`.
2. Знайдіть суму елементів послідовності цілих чисел, що є кратними числу 5 і завершується числом 0.

Варіант 13:

1. Обчисліть значення функції $y = |x|$ на відрізку $[-5; 5]$ з кроком 0.1 за допомогою циклу `while`.
2. Визначте найбільший елемент послідовності дійсних чисел, що завершується числом 0.

Варіант 14:

1. Обчисліть значення функції $y = \sin^2(x)$ на відрізку $[0; \pi]$ з кроком 0.1 за допомогою циклу `for`.
2. Знайдіть добуток елементів послідовності натуральних чисел, що є кратними числу 7 і завершується числом 0.

Варіант 15:

1. Обчисліть значення функції $y = \cos^2(x)$ на відрізку $[0; \pi]$ з кроком 0.1 за допомогою циклу `while`.
2. Визначте найменший елемент послідовності дійсних чисел, що завершується числом 0.

Варіант 16:

1. Обчисліть значення функції $y = \tan^2(x)$ на відрізку $[0; \pi/4]$ з кроком 0.1 за допомогою циклу for.
2. Знайдіть середнє геометричне елементів послідовності натуральних чисел, що завершується числом 0.

Варіант 17:

1. Обчисліть значення функції $y = e^{(2x)}$ на відрізку $[0; 1]$ з кроком 0.1 за допомогою циклу while.
2. Визначте кількість елементів послідовності цілих чисел, що є кратними числу 9 і завершується числом 0.

Варіант 18:

1. Обчисліть значення функції $y = x^4$ на відрізку $[0; 3]$ з кроком 0.1 за допомогою циклу for.
2. Знайдіть добуток парних елементів послідовності натуральних чисел, що завершується числом 0.

Варіант 19:

1. Обчисліть значення функції $y = 2^{(2x)}$ на відрізку $[0; 3]$ з кроком 0.1 за допомогою циклу while.
2. Визначте кількість елементів послідовності цілих чисел, що є кратними числу 4 і завершується числом 0.

Варіант 20:

1. Обчисліть значення функції $y = \log_2(x)$ на відрізку $[1; 10]$ з кроком 0.1 за допомогою циклу for.
2. Знайдіть найбільший непарний елемент послідовності натуральних чисел, що завершується числом 0.

Варіант 21:

1. Обчисліть значення функції $y = |x|^3$ на відрізку $[-2; 2]$ з кроком 0.1 за допомогою циклу while.
2. Знайдіть добуток елементів послідовності цілих чисел, що є кратними числу 6 і завершується числом 0.

Варіант 22:

1. Обчисліть значення функції $y = \sin(x)/x$ на відрізку $[1; 10]$ з кроком 0.1 за допомогою циклу `for`.
2. Визначте найменший парний елемент послідовності натуральних чисел, що завершується числом 0.

Варіант 23:

1. Обчисліть значення функції $y = x^{(1/2)}$ на відрізку $[0; 10]$ з кроком 0.1 за допомогою циклу `while`.
2. Визначте найбільший елемент послідовності дійсних чисел, що завершується числом 0.

Варіант 24:

1. Обчисліть значення функції $y = \cos(x)/x$ на відрізку $[1; 10]$ з кроком 0.1 за допомогою циклу `for`.
2. Знайдіть добуток елементів послідовності натуральних чисел, що є кратними числу 8 і завершується числом 0.

Варіант 25:

1. Обчисліть значення функції $y = e^{(x^2)}$ на відрізку $[0; 2]$ з кроком 0.1 за допомогою циклу `while`.
2. Визначте найменший елемент послідовності дійсних чисел, що завершується числом 0.

Контрольні запитання

1. Що таке цикл в програмуванні? Яка його роль?
2. Що таке ітерація?
3. Які типи циклів існують в Python? Як вони використовуються?
4. Як працює цикл `for` в Python? Які в нього є особливості?
5. Як працює цикл `while` в Python? Які в нього є особливості?
6. Що таке оператори `break` і `continue`? Як вони використовуються в Python?
7. Як можна перебрати елементи списку в Python за допомогою циклу?

ПРАКТИЧНА РОБОТА №4: СИСТЕМИ ЧИСЛЕННЯ

Мета роботи – засвоїти принципи переведення чисел в різні системи числення.

Теоретичні відомості

Система числення – це спосіб представлення чисел, який використовує набір символів (цифр) та базу (або основу) для представлення чисел. Найбільш відомі системи числення – це десяткова (база 10), двійкова (база 2), вісімкова (база 8) і шістнадцяткова (база 16).

Десяткова система – це система числення, що має 10 цифр (від 0 до 9) та основу 10.

Двійкова система – це система числення, яка має всього дві цифри - 0 і 1. Це основний мова комп'ютерів.

Вісімкова і шістнадцяткова системи – це системи числення, які мають відповідно 8 і 16 цифр. Вони часто використовуються в комп'ютерних науках, оскільки вони є множниками 2 і легко переводяться в двійкову систему і назад.

Переведення між системами числення – це важливий навик в інформатиці.

Переведення цілих чисел між системами числення.

Приклад переведення числа з десятичної системи в двійкову:

- Запишемо число 13 в десятичній системі.
- Ділимо число на 2, записуємо остачу. $13 / 2 = 6$ (остача 1).
- Ділимо отримане число на 2, записуємо остачу. $6 / 2 = 3$ (остача 0).
- Продовжуємо ділити на 2, доки не отримаємо 0. $3 / 2 = 1$ (остача 1), $1 / 2 = 0$ (остача 1).
- Записуємо остачі у зворотному порядку. Отримаємо число 1101 в двійковій системі.

Переведення дробових чисел між системами числення. Приклад переведення дробового числа з десятичної системи в двійкову:

- Розглянемо дробове число 0.625 в десятковій системі.
- Помножимо дробову частину числа на 2, запишемо цілу частину.
 $0.625 * 2 = 1.25$ (запишемо 1).
- Помножимо дробову частину отриманого числа на 2, запишемо цілу частину.
 $0.25 * 2 = 0.5$ (запишемо 0).
- Продовжуємо множити дробову частину на 2, поки не отримаємо 0 або поки не отримаємо потрібну кількість знаків після коми.
 $0.5 * 2 = 1.0$ (запишемо 1).
- Запишемо отримані числа. Отримаємо число 0.101 в двійковій системі.

Переведення дробового двійкового числа до десяткового. Розглянемо число 1101.101 в двійковій системі. Воно складається з цілої частини 1101 та дробової частини 101. Перетворимо кожну частину окремо:

- Ціла частина: починаючи з правої сторони, множимо кожну цифру на 2, підняте в степінь, який відповідає її позиції (починаючи з 0). Додаємо отримані числа:
 $1*2^0 + 0*2^1 + 1*2^2 + 1*2^3 = 1 + 0 + 4 + 8 = 13$.
- Дробова частина: починаючи з лівої сторони, множимо кожну цифру на 2, підняте в степінь, який відповідає її позиції (від'ємні степені, починаючи з -1). Додаємо отримані числа:
 $1*2^{-1} + 0*2^{-2} + 1*2^{-3} = 0.5 + 0 + 0.125 = 0.625$.

Отже, число 1101.101 в двійковій системі перетворюється в число 13.625 в десятковій системі.

Переведення з двійкової до вісімкової (в зворотному напрямі аналогічні дії):

- Розбиваємо двійкове число на групи по три біти, починаючи справа: 101 100 101.
- Перетворюємо кожну групу в її вісімковий еквівалент: 5 4 5.
- Таким чином, число 101100101 в двійковій системі перетворюється в число 545 в вісімковій системі.

Переведення з двійкової до шістнадцяткової (в зворотному напрямі аналогічні дії):

- Розбиваємо двійкове число на групи по чотири біти, починаючи справа: 1101 0101.
- Перетворюємо кожну групу в її шістнадцятковий еквівалент: D A.
- Таким чином, число 11010101 в двійковій системі перетворюється в число DA в шістнадцятковій системі.

Практична частина

Зауваження: виконання завдань 1-2 кожного варіанту повинно містити покроковий алгоритм, як завдання виконувалося, а не лише остаточну відповідь. Для всіх варіантів третє завдання – зробити перші два завдання за допомогою Python.

Варіант 1:

1. Перевести число 101101 з двійкової системи в десяткову та шістнадцяткову системи.
2. Перевести число 5.75 з десяткової системи в двійкову та число 101.11 (2) з двійкової в десяткову.

Варіант 2:

1. Перевести число 7A з шістнадцяткової системи в десяткову та двійкову системи.
2. Перевести число 2.875 з десяткової системи в двійкову та число 10.111 (2) з двійкової в десяткову.

Варіант 3:

1. Перевести число 256 з десяткової системи в двійкову та шістнадцяткову системи.
2. Перевести число 7.125 з десяткової системи в двійкову та число 111.001 (2) з двійкової в десяткову.

Варіант 4:

1. Перевести число 11111111 з двійкової системи в десяткову та шістнадцяткову системи.

2. Перевести число 9.375 з десяткової системи в двійкову та число 1001.011 (2) з двійкової в десяткову.

Варіант 5:

1. Перевести число 89 з десяткової системи в двійкову та шістнадцяткову системи.
2. Перевести число 6.5 з десяткової системи в двійкову та число 110.1 (2) з двійкової в десяткову.

Варіант 6:

1. Перевести число 10101010 з двійкової системи в десяткову та шістнадцяткову системи.
2. Перевести число 4.75 з десяткової системи в двійкову та число 100.11 (2) з двійкової в десяткову.

Варіант 7:

1. Перевести число FF з шістнадцяткової системи в десяткову та двійкову системи.
2. Перевести число 8.375 з десяткової системи в двійкову та число 1000.011 (2) з двійкової в десяткову.

Варіант 8:

1. Перевести число 365 з десяткової системи в двійкову та шістнадцяткову системи.
2. Перевести число 3.25 з десяткової системи в двійкову та число 11.01 (2) з двійкової в десяткову.

Варіант 9:

1. Перевести число ABC з шістнадцяткової системи в десяткову та двійкову системи.
2. Перевести число 2.625 з десяткової системи в двійкову та число 10.101 (2) з двійкової в десяткову.

Варіант 10:

1. Перевести число 10011001 з двійкової системи в десяткову та шістнадцяткову системи.

2. Перевести число 7.375 з десяткової системи в двійкову та число 111.011 (2) з двійкової в десяткову.

Варіант 11:

1. Перевести число 500 з десяткової системи в двійкову та шістнадцяткову системи.
2. Перевести число 5.125 з десяткової системи в двійкову та число 101.001 (2) з двійкової в десяткову.

Варіант 12:

1. Перевести число SAFE з шістнадцяткової системи в десяткову та двійкову системи.
2. Перевести число 9.5 з десяткової системи в двійкову та число 1001.1 (2) з двійкової в десяткову.

Варіант 13:

1. Перевести число 1234 з десяткової системи в двійкову та шістнадцяткову системи.
2. Перевести число 6.25 з десяткової системи в двійкову та число 110.01 (2) з двійкової в десяткову.

Варіант 14:

1. Перевести число BEAD з шістнадцяткової системи в десяткову та двійкову системи.
2. Перевести число 3.875 з десяткової системи в двійкову та число 11.111 (2) з двійкової в десяткову.

Варіант 15:

1. Перевести число 10110011 з двійкової системи в десяткову та шістнадцяткову системи.
2. Перевести число 4.625 з десяткової системи в двійкову та число 100.101 (2) з двійкової в десяткову.

Варіант 16:

1. Перевести число 1987 з десяткової системи в двійкову та шістнадцяткову системи.

2. Перевести число 8.75 з десяткової системи в двійкову та число 1000.11 (2) з двійкової в десяткову.

Варіант 17:

1. Перевести число FACE з шістнадцяткової системи в десяткову та двійкову системи.
2. Перевести число 5.25 з десяткової системи в двійкову та число 101.01 (2) з двійкової в десяткову.

Варіант 18:

1. Перевести число 11001100 з двійкової системи в десяткову та шістнадцяткову системи.
2. Перевести число 7.5 з десяткової системи в двійкову та число 111.1 (2) з двійкової в десяткову.

Варіант 19:

1. Перевести число 2023 з десяткової системи в двійкову та шістнадцяткову системи.
2. Перевести число 2.75 з десяткової системи в двійкову та число 10.01 (2) з двійкової в десяткову.

Варіант 20:

1. Перевести число DEAD з шістнадцяткової системи в десяткову та двійкову системи.
2. Перевести число 3.5 з десяткової системи в двійкову та число 11.101 (2) з двійкової в десяткову.

Варіант 21:

1. Перевести число 11110000 з двійкової системи в десяткову та шістнадцяткову системи.
2. Перевести число 6.875 з десяткової системи в двійкову та число 1001.11 (2) з двійкової в десяткову.

Варіант 22:

1. Перевести число 789 з десяткової системи в двійкову та шістнадцяткову системи.

2. Перевести число 9.625 з десяткової системи в двійкову та число 1101.01 (2) з двійкової в десяткову.

Варіант 23:

1. Перевести число BEEF з шістнадцяткової системи в десяткову та двійкову системи.
2. Перевести число 5.375 з десяткової системи в двійкову та число 1010.101 (2) з двійкової в десяткову.

Варіант 24:

1. Перевести число 1001 з двійкової системи в десяткову та шістнадцяткову системи.
2. Перевести число 8.125 з десяткової системи в двійкову та число 110.01 (2) з двійкової в десяткову.

Варіант 25:

1. Перевести число 314 з десяткової системи в двійкову та шістнадцяткову системи.
2. Перевести число 7.625 з десяткової системи в двійкову та число 101.101 (2) з двійкової в десяткову.

Контрольні запитання

1. Що таке система числення?
2. Назвіть основні системи числення.
3. Які особливості двійкової системи числення?
4. Які особливості вісімкової системи числення?
5. Які особливості шістнадцяткової системи числення?
6. Що таке основа системи числення?
7. Як переводить число з однієї системи числення в іншу?
8. Як перевести число з двійкової системи в десяткову?

ПРАКТИЧНА РОБОТА №5: СПИСКИ У PYTHON

Мета роботи – Вдосконалити навички роботи зі списками в Python, ознайомитись з різними методами і операціями, які можна виконувати над списками, навчитися вирішувати практичні завдання, пов'язані з обробкою списків.

Теоретичні відомості

Списки в Python – це впорядковані колекції об'єктів. Об'єкти, що знаходяться в списку, називаються його елементами або пунктами. Списки можуть містити елементи будь-якого типу, і елементи в одному списку не обов'язково мають бути одного типу.

Список створюється, вкладаючи послідовність елементів в квадратні дужки [], розділяючи їх комами. Наприклад, список цілих чисел можна створити так:

```
numbers = [1, 2, 3, 4, 5]
```

Можна створити список з елементами різних типів:

```
mixed_list = [1, "apple", 3.14, ["another", "list"]]
```

Останній елемент цього списку сам є списком, тому ми говоримо, що списки можуть бути вкладеними.

Індексація списків. Елементи списку доступні через індекси. Індексація починається з нуля для першого елемента. Так, `numbers[0]` верне 1. Індекси можуть бути також від'ємними, в цьому випадку вони відлік ведеться з кінця списку, тобто `numbers[-1]` верне останній елемент списку.

Зрізи списків. Зрізи дозволяють вам отримати підсписок з декількох елементів. Синтаксис зрізів - це використання : в операторі індексації. Наприклад, `numbers[1:3]` верне список [2, 3].

Методи списків. Python включає в себе декілька корисних методів для роботи зі списками. Ось декілька прикладів:

- `append(x)`: Додає елемент `x` в кінець списку.
- `extend(L)`: Додає всі елементи списку `L` в кінець списку.
- `insert(i, x)`: Вставляє елемент `x` в позицію `i`.
- `remove(x)`: Видаляє перший елемент, який є рівним `x`.
- `pop(i)`: Видаляє елемент, що знаходиться в позиції `i`, і повертає його.
- `index(x)`: Повертає індекс першого елемента, що є рівним `x`.
- `count(x)`: Повертає кількість разів, коли `x` зустрічається в списку.
- `sort()`: Сортує елементи списку.
- `reverse()`: Перевертає порядок елементів в списку.

Спискові включення (list comprehensions). Спискові включення – це спосіб створювати нові списки, використовуючи існуючий список. Вони можуть бути особливо корисними для створення списків, які відповідають певному критерію або що піддаються певному обчисленню. Наприклад, щоб створити список квадратів чисел від 1 до 5, можна використати спискове включення таким чином:

```
squares = [x**2 for x in numbers]
```

Цей код створює новий список, в якому кожен елемент - це квадрат відповідного елемента зі списку `numbers`.

Практична частина

Зауваження: у завдання забороняється використовувати методи циклу або вбудовані функції. Тільки цикли або логічні вирази.

Варіант 1:

1. Вихідний список: `[5, 10, 15, 20, 25, 30, 35, 40, 45, 50]`. Знайдіть середнє геометричне всіх елементів списку.
2. Вихідний список: `[1, 2, 3, 4, 5, 6, 7, 8, 9]`. Знайдіть куби всіх елементів списку.

Варіант 2:

1. Вихідний список: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]. Знайдіть факторіали всіх елементів списку.
2. Вихідний список: [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]. Порахуйте, скільки чисел у списку є простими числами.

Варіант 3:

1. Вихідний список: [5, 7, 11, 13, 17, 19, 23, 29, 31, 37]. Перевірте, чи всі числа у списку є простими числами.
2. Вихідний список: [1, 1, 2, 3, 5, 8, 13, 21, 34, 55]. Перевірте, чи всі числа у списку є числами Фібоначчі.

Варіант 4:

1. Вихідний список: [1, 1, 2, 2, 3, 3, 4, 4, 5, 5]. Видаліть всі дублікати в списку.
2. Вихідний список: [5, 4, 3, 2, 1]. Відсортуйте список у порядку зростання.

Варіант 5:

1. Вихідний список: [1, 2, 3, 4, 5, 6, 7, 8, 9]. Перевірте, чи є список строго зростаючим.
2. Вихідний список: [9, 8, 7, 6, 5, 4, 3, 2, 1]. Перевірте, чи є список строго спадним.

Варіант 6:

1. Вихідний список: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. Перевірте, чи є список монотонним.
2. Вихідний список: [1, 2, 3, 4, 5]. Знайдіть медіану списку.

Варіант 7:

1. Вихідний список: [1, 2, 3, 4, 5, 6, 7, 8, 9]. Знайдіть моду списку.
2. Вихідний список: [1, 2, 3, 4, 5]. Визначте, чи є список паліндромом.

Варіант 8:

1. Вихідний список: [1, 2, 3, 4, 5]. Оберніть список.
2. Вихідний список: [1, 2, 3, 4, 5]. Замініть 3-й елемент списку на 10.

Варіант 9:

1. Вихідний список: [1, 2, 3, 4, 5]. Поміняйте перший і останній елементи місцями.
2. Вихідний список: [1, 2, 3, 4, 5]. Вставте елемент 6 на 3-є місце списку.

Варіант 10:

1. Вихідний список: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. Порахуйте, скільки парних і непарних чисел у списку.
2. Вихідний список: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. Знайдіть середнє квадратичне всіх елементів списку.

Варіант 11:

1. Вихідний список: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. Визначте, скільки у списку елементів, більших за середнє арифметичне.
2. Вихідний список: [2, 3, 4, 5, 6, 7, 8, 9, 10, 11]. Знайдіть всі дільники кожного числа в списку.

Варіант 12:

1. Вихідний список: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. Знайдіть всі числа, які діляться на 3 без остачі.
2. Вихідний список: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. Знайдіть всі числа, які діляться на 5 без остачі.

Варіант 13:

1. Вихідний список: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. Видаліть всі числа, які діляться на 3 без остачі.
2. Вихідний список: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. Видаліть всі числа, які діляться на 5 без остачі.

Варіант 14:

1. Вихідний список: [1, 2, 3, 4, 5]. Знайдіть різницю між максимальним та мінімальним елементами.
2. Вихідний список: [1, 2, 3, 4, 5]. Знайдіть різницю між середнім арифметичним та середнім геометричним елементами.

Варіант 15:

1. Вихідний список: [1, 2, 3, 4, 5]. Замініть кожний елемент списку на його квадрат.
2. Вихідний список: [1, 2, 3, 4, 5]. Замініть кожний елемент списку на його куб.

Варіант 16:

1. Вихідний список: [1, 2, 3, 4, 5]. Знайдіть суму елементів списку.
2. Вихідний список: [1, 2, 3, 4, 5]. Знайдіть добуток елементів списку.

Варіант 17:

1. Вихідний список: [1, 2, 3, 4, 5]. Знайдіть найбільший елемент списку.
2. Вихідний список: [1, 2, 3, 4, 5]. Знайдіть найменший елемент списку.

Варіант 18:

1. Вихідний список: [1, 2, 3, 4, 5]. Порахуйте кількість входжень кожного елемента в список.
2. Вихідний список: [1, 2, 3, 4, 5, 2, 2, 4, 4, 5]. Знайдіть елемент, що зустрічається найчастіше.

Варіант 19:

1. Вихідний список: [1, 2, 3, 4, 5]. Видаліть 3-й елемент списку.
2. Вихідний список: [1, 2, 3, 4, 5]. Видаліть останній елемент списку.

Варіант 20:

1. Вихідний список: [1, 2, 3, 4, 5]. Вставте елемент 6 на 3-є місце списку.
2. Вихідний список: [1, 2, 3, 4, 5]. Вставте елемент 0 на початок списку.

Варіант 21:

1. Вихідний список: [1, 2, 3, 4, 5]. Перевірте, чи є в списку елемент 3.
2. Вихідний список: [1, 2, 3, 4, 5]. Перевірте, чи є в списку елемент 6.

Варіант 22:

1. Вихідний список: [1, 2, 3, 4, 5]. Знайдіть середнє арифметичне елементів списку.
2. Вихідний список: [1, 2, 3, 4, 5]. Знайдіть середнє геометричне елементів списку.

Варіант 23:

1. Вихідний список: [1, 2, 3, 4, 5]. Знайдіть медіану списку.
2. Вихідний список: [1, 2, 3, 4, 5]. Знайдіть моду списку.

Варіант 24:

1. Вихідний список: [1, 2, 3, 4, 5]. Оберніть порядок елементів в списку.
2. Вихідний список: [1, 2, 3, 4, 5]. Зробіть список паліндромом, додаючи елементи у зворотному порядку.

Варіант 25:

1. Вихідний список: [1, 2, 3, 4, 5]. Скопіюйте список.
2. Вихідний список: [1, 2, 3, 4, 5]. Створіть новий список, що містить тільки унікальні елементи вихідного списку

Контрольні запитання

1. Що таке список в Python?
2. Як створити список в Python?
3. Що таке індексація списків? Як вона працює?
4. Що таке зріз списку? Як створити зріз списку?
5. Що таке вкладені списки? Дайте приклад вкладеного списку.
6. Що таке спискові включення (list comprehensions)? Дайте приклад.
7. Як копіювати списки в Python? Яка різниця між поверхневим і глибоким копіюванням.

ПРАКТИЧНА РОБОТА №6: РЯДКИ У PYTHON

Мета роботи – набуття практичних навичок в роботі з рядками та методами рядків в Python.

Теоретичні відомості

Рядки в Python – це незмінні послідовності символів. Ми можемо використовувати одинарні або подвійні лапки для створення рядків. Python використовує Unicode для представлення символів у рядках.

Створення рядків

Створити рядок в Python можна так:

```
s = 'Hello, world!'
s = "Hello, world!"
```

Операції з рядками

Основні операції над рядками включають конкатенацію (об'єднання), повторення, індексування, зрізи і перевірку наявності:

```
s = 'Hello, ' + 'world!' # конкатенація
s = 'Hello, ' * 3 # повторення
s = 'Hello, world!'
print(s[0]) # індексування
print(s[0:5]) # зріз
print('Hello' in s) # перевірка наявності
```

Методи рядків

Python має багато вбудованих методів для рядків, що дають нам можливість виконувати операції з рядками. Деякі з них включають:

- `s.split(sep=None)`: розбиває рядок на список рядків за допомогою `sep` як роздільника.

- `s.join(iterable)`: об'єднує елементи `iterable` в один рядок з `s` як роздільником.
- `s.replace(old, new[, count])`: замінює всі входження `old` на `new` в рядку. Якщо вказано `count`, замінюється лише перші `count` входження.
- `s.find(sub[, start[, end]])`: повертає індекс першого входження `sub` в рядок або `-1`, якщо `sub` не знайдено.
- `s.count(sub[, start[, end]])`: рахує кількість входжень `sub` в рядок.

Форматування рядків

Python підтримує кілька способів форматування рядків. Ми можемо використовувати оператор `%`, метод `format()` або `f-рядки`:

```
name = 'world'
print('Hello, %s!' % name) # оператор %
print('Hello, {}'.format(name)) # метод format
print(f'Hello, {name}!') # f-рядок
```

Практична частина

Варіант 1:

1. "Aston approach is an evolution of prior conversations about 'broad', 'narrow', 'strong' and 'weak' forms of AI.". Підрахуйте кількість ком, лапок і крапок у рядку.
2. "Advancements in technology have made it possible to utilize machine learning in many different industries.". Знайдіть і виведіть усі числа з рядка.

Варіант 2:

1. "Artificial general intelligence (AGI) refers to highly autonomous systems.". Розділіть рядок на слова і виведіть усі слова, що починаються на 'a' (велика або маленька буква).
2. "Deep learning is a subset of machine learning, which is essentially a neural network with three or more layers.". Замініть усі пробіли на дефіси.

Варіант 3

1. "These layers are known as hidden layers because they do not directly interact with the input or output.". Знайдіть довжину найдовшого та найкоротшого слова у рядку.
2. "Deep learning models are built using neural network architectures, and neural networks have the ability to automatically learn representations for data by training themselves.". Видаліть усі голосні букви з рядка.

Варіант 4:

1. "A common example of a neural network architecture is the multilayer perceptron (MLP)". Перевірте, чи є рядок паліндромом (розглядаючи тільки літери і цифри, без урахування регістру).
2. "This includes input and output layers, as well as one or more hidden layers.". Знайдіть і виведіть усі слова, що закінчуються на 's'.

Варіант 5:

1. "The term deep learning was introduced to the machine learning community by Rina Dechter in 1986, and to artificial neural networks by Igor Aizenberg and colleagues in 2000.". Знайдіть кількість входжень слова 'learning' в рядку.
2. "In statistics, the logistic model (or logit model) is used to model the probability of a certain class or event.". Перевірте, чи є рядок ASCII (В Python, рядок вважається ASCII, якщо кожний символ в рядку може бути представлений за допомогою ASCII кодування. ASCII кодування покриває символи в діапазоні від 0 до 127 включно).

Варіант 6:

1. "The limits of my language mean the limits of my world.". Виведіть всі слова, що містять більше ніж 5 літер.
2. "The limits of my language mean the limits of my world.". Замініть "world" на "universe".

Варіант 7:

1. "There is no abstract art. You must always start with something.". Видаліть всі прийменники.
2. "There is no abstract art. You must always start with something.". Перетворіть всі перші букви слів на великі.

Варіант 8:

1. "The world is full of obvious things which nobody by any chance ever observes.". Підрахуйте кількість входжень "of" у рядку.
2. "The world is full of obvious things which nobody by any chance ever observes.". Перевірте, чи є рядок паліндромом.

Варіант 9:

1. "If you want a happy ending, that depends, of course, on where you stop your story.". Виведіть всі слова, що починаються на 's'.
2. "If you want a happy ending, that depends, of course, on where you stop your story.". Видаліть всі слова, що закінчуються на 's'.

Варіант 10:

1. "Success is not the key to happiness. Happiness is the key to success. If you love what you are doing, you will be successful.". Замініть "Success" на "Achievement".
2. "Success is not the key to happiness. Happiness is the key to success. If you love what you are doing, you will be successful.". Розділіть рядок на список слів.

Варіант 11:

1. "It's not what happens to you, but how you react to it that matters.". Перетворіть весь рядок на малі літери.
2. "It's not what happens to you, but how you react to it that matters.". Перевірте, чи всі символи в рядку є буквами.

Варіант 12:

1. "Try not to become a man of success, but rather try to become a man of value.". Виведіть всі слова, що закінчуються на 's'.

2. "Try not to become a man of success, but rather try to become a man of value.". Замініть 'man' на 'person'.

Варіант 13:

1. "The only way to achieve the impossible is to believe it is possible.". Перетворіть весь рядок на великі літери.
2. "The only way to achieve the impossible is to believe it is possible.". Виведіть всі слова, що мають рівно 4 літери.

Варіант 14:

1. "Success is not in what you have, but who you are.". Видаліть всі коми з рядка.
2. "Success is not in what you have, but who you are.". Замініть всі пробіли на дефіси.

Варіант 15:

1. "Time is a created thing. To say 'I don't have time,' is like saying, 'I don't want to.'". Замініть всі апострофи на цитатні знаки.
2. "Time is a created thing. To say 'I don't have time,' is like saying, 'I don't want to.'". Розділіть рядок на два рядки по першому реченню.

Варіант 16:

1. "People with courage and character always seem to have the beautiful birds on their side.". Виведіть всі слова, що починаються на 'c'.
2. "People with courage and character always seem to have the beautiful birds on their side.". Перевірте, чи є рядок ASCII.

Варіант 17:

1. "Remember, your work may be only to sweep the sidewalk, but it's the way you do it that gives satisfaction.". Виведіть всі слова, що закінчуються на 's'.
2. "Remember, your work may be only to sweep the sidewalk, but it's the way you do it that gives satisfaction.". Замініть 'satisfaction' на 'contentment'.

Варіант 18:

1. "Don't aim for success if you want it; just do what you love and believe in, and it will come naturally.". Замініть всі крапки на коми.
2. "Don't aim for success if you want it; just do what you love and believe in, and it will come naturally.". Видаліть всі слова, що починаються на 'a'.

Варіант 19:

1. "Success is not just making money. It's making a difference.". Перетворіть весь рядок на малі літери.
2. "Success is not just making money. It's making a difference.". Замініть всі вхождення 'a' на 'the'.

Варіант 20:

1. "Every day may not be good, but there's something good in every day.". Виведіть всі слова, що мають рівно 3 літери.
2. "Every day may not be good, but there's something good in every day.". Перетворіть весь рядок на великі літери.

Варіант 21:

1. "Success is not the key to happiness. Happiness is the key to success. If you love what you are doing, you will be successful.". Замініть "Success" на "Achievement".
2. "Success is not the key to happiness. Happiness is the key to success. If you love what you are doing, you will be successful.". Розділіть рядок на список слів.

Варіант 22:

1. "It's not what happens to you, but how you react to it that matters.". Перетворіть весь рядок на малі літери.
2. "It's not what happens to you, but how you react to it that matters.". Перевірте, чи всі символи в рядку є буквами.

Варіант 23:

1. "Try not to become a man of success, but rather try to become a man of value.". Виведіть всі слова, що закінчуються на 's'.

2. "Try not to become a man of success, but rather try to become a man of value.". Замініть 'man' на 'person'.

Варіант 24:

1. "The only way to achieve the impossible is to believe it is possible.". Перетворіть весь рядок на великі літери.
2. "The only way to achieve the impossible is to believe it is possible.". Виведіть всі слова, що мають рівно 4 літери.

Варіант 25:

1. "Success is not in what you have, but who you are.". Видаліть всі коми з рядка.
2. "Success is not in what you have, but who you are.". Замініть всі пробіли на дефіси.

Контрольні запитання

1. Що таке рядки в Python і як їх створити?
2. Які основні операції можна виконати з рядками в Python?
3. Опишіть наведені методи рядків в Python: `split()`, `join()`, `replace()`, `find()`, `count()`.
4. Як можна форматовувати рядки в Python? Наведіть приклади.

ПРАКТИЧНА РОБОТА №7: СЛОВНИКИ ТА МНОЖИНИ У PYTHON

Мета роботи – засвоїти навички роботи із словниками та множинами в Python, ознайомитись з різними методами і операціями, які можна виконувати, навчитися вирішувати практичні завдання, пов'язані з обробкою словників та множин.

Теоретичні відомості

Словники у Python.

Словник в Python – це вбудований тип даних, що має пари "ключ-значення". Словники в Python незмінні, а їхні ключі мають унікальні значення. Приклад створення словника:

```
dict1 = {'key1': 'value1', 'key2': 'value2', 'key3': 'value3'}
```

Щоб отримати значення за ключем, можна використати наступний синтаксис:

```
print(dict1['key2']) # виведе: value2
```

Щоб додати нову пару "ключ-значення":

```
dict1['key4'] = 'value4'
```

Щоб видалити пару "ключ-значення", використовуйте команду del:

```
del dict1['key1']
```

Щоб перевірити наявність ключа в словнику, зазвичай використовується оператор in:

```
print('key2' in dict1) # виведе: True
```

Множини у Python.

Множина в Python – це вбудований тип даних, що містить унікальні елементи. Вони схожі на списки або словники, але їх не можна індексувати.

Створити множину можна за допомогою функції `set()`, або за допомогою фігурних дужок `{}`:

```
set1 = set([1, 2, 3, 4, 5])
```

```
set2 = {1, 2, 2, 3, 4} # автоматично видалить дублікати, результат  
буде {1, 2, 3, 4}
```

Щоб додати елемент до множини, зазвичай використовується метод `add()`:

```
set1.add(6)
```

Щоб видалити елемент з множини, метод `remove()`:

```
set1.remove(1)
```

Множини підтримують математичні операції, такі як об'єднання (`union`), перетин (`intersection`), різниця (`difference`), та симетрична різниця (`symmetric_difference`):

```
set1 = {1, 2, 3, 4}
```

```
set2 = {3, 4, 5, 6}
```

```
print(set1.union(set2)) # виведе: {1, 2, 3, 4, 5, 6}
```

```
print(set1.intersection(set2)) # виведе: {3, 4}
```

```
print(set1.difference(set2)) # виведе: {1, 2}
```

```
print(set1.symmetric_difference(set2)) # виведе: {1, 2, 5, 6}
```

Практична частина

Зауваження: у завданнях, де дано словник, або множину, мається на увазі, що значення для їх створення або вводяться з клавіатури користувачем, або зчитуються з файлу, або встановлені за замовчуванням.

Варіант 1:

1. Словники: Дано словник: `student = {'name': 'John', 'age': 25, 'courses': ['Math', 'CompSci']}`. Змініть ім'я студента на 'Jane' і додайте новий курс 'Physics'.
2. Множини: Напишіть програму, яка приймає два списки і виводить всі елементи, які є у обох списках.

Варіант 2:

1. Словники: Створіть словник, який містить інформацію про вас: ім'я, вік, улюблені фільми. Потім додайте в словник список ваших улюблених книг.
2. Множини: Створіть множину з рядків 'hello', 'world', 'python', а потім додайте до цієї множини рядок 'programming'.

Варіант 3:

1. Словники: Напишіть програму, яка бере словник з іменами і віком людей, і виводить імена всіх людей, які старші 18 років.
2. Множини: Напишіть програму, яка приймає список чисел і виводить кількість різних чисел в цьому списку.

Варіант 4:

1. Словники: Створіть словник, в якому ключі - це числа від 1 до 15, а значення - це квадрати цих чисел.
2. Множини: Напишіть програму, яка приймає дві множини і виводить всі елементи, які є в одній множині, але відсутні в іншій.

Варіант 5:

1. Словники: Напишіть програму, яка обчислює кількість різних букв в рядку (регістр не має значення).
2. Множини: Створіть множину з чисел від 1 до 10, а потім видаліть з множини числа 3, 5 і 7.

Варіант 6:

1. Словники: Дано словник з іменами та балами студентів. Знайдіть студента з найвищим балом.

2. Множини: Напишіть програму, яка приймає дві множини чисел і виводить всі числа, які є в обох множинах.

Варіант 7:

1. Словники: Створіть словник, де ключами є числа від 1 до 10, а значеннями - їх куби.
2. Множини: Створіть множину з чисел від 1 до 20, а потім видаліть з множини всі числа, які діляться на 3.

Варіант 8:

1. Словники: Напишіть програму, яка об'єднує два словники в один.
2. Множини: Напишіть програму, яка приймає список чисел і виводить число, яке зустрічається в списку найчастіше.

Варіант 9:

1. Словники: Створіть словник з інформацією про книгу: автор, назва, рік видання. Змініть рік видання на поточний.
2. Множини: Напишіть програму, яка приймає дві множини і виводить всі числа, які є в одній множині, але відсутні в іншій.

Варіант 10:

1. Словники: Дано словник з іменами та віком людей. Видаліть зі словника людей, вік яких менше 21 року.
2. Множини: Створіть множину з рядків 'hello', 'world', 'python', а потім видаліть з множини рядок 'world'.

Варіант 11:

1. Словники: Напишіть програму, яка виводить всі ключі словника, значення яких є парними числами.
2. Множини: Напишіть програму, яка приймає список чисел і виводить число, яке зустрічається в списку найрідше.

Варіант 12:

1. Словники: Дано словник, де ключами є слова, а значеннями - їх переклади. Напишіть програму, яка перекладає речення, задане списком слів.

2. Множини: Напишіть програму, яка приймає дві множини і виводить всі елементи, які є в обох множинах.

Варіант 13:

1. Словники: Створіть словник з інформацією про пісню: виконавець, назва, тривалість. Додайте до словника жанр пісні.
2. Множини: Створіть множину з чисел від 1 до 10, а потім видаліть з множини всі непарні числа.

Варіант 14:

1. Словники: Напишіть програму, яка приймає словник з іменами та оцінками студентів і виводить середню оцінку кожного студента.
2. Множини: Створіть множину з рядків 'apple', 'banana', 'cherry', а потім додайте до множини рядок 'orange'.

Варіант 15:

1. Словники: Створіть словник, де ключами є імена людей, а значеннями - їх улюблені кольори. Змініть улюблений колір однієї з людей.
2. Множини: Напишіть програму, яка приймає список чисел і виводить кількість різних чисел в цьому списку.

Варіант 16:

1. Словники: Створіть словник, в якому ключами є числа від 1 до 10, а значеннями - їх квадрати.
2. Множини: Напишіть програму, яка приймає множину чисел і виводить суму всіх чисел множини.

Варіант 17:

1. Словники: Напишіть програму, яка приймає словник з іменами та віком студентів і виводить ім'я найстаршого студента.
2. Множини: Створіть множину з чисел від 1 до 10, а потім додайте до множини всі числа від 11 до 20.

Варіант 18:

1. Словники: Дано словник з іменами та балами студентів. Знайдіть студента з найнижчим балом.

2. Множини: Напишіть програму, яка приймає дві множини і виводить всі елементи, які є в одній множині, але не є в обох множинах одночасно.

Варіант 19:

1. Словники: Дано словник з іменами та балами студентів. Знайдіть середній бал.
2. Множини: Створіть множину з рядків 'apple', 'banana', 'cherry', а потім перевірте, чи є 'banana' в множині.

Варіант 20:

1. Словники: Дано словник з країнами та їх столицями. Виведіть всі країни.
2. Множини: Напишіть програму, яка приймає список чисел і виводить список, що містить лише унікальні числа даного списку.

Варіант 21:

1. Словники: Дано словник з країнами та їх столицями. Виведіть всі столиці.
2. Множини: Дано дві множини. Виведіть всі елементи, які є в обох множинах.

Варіант 22:

1. Словники: Дано словник з іменами та балами студентів. Додайте до словника нового студента.
2. Множини: Дано дві множини. Виведіть всі елементи, які є в першій множині, але не є в другій.

Варіант 23:

1. Словники: Дано словник з іменами та балами студентів. Видаліть із словника студента з найнижчим балом.
2. Множини: Дано дві множини. Виведіть всі елементи, які є в другій множині, але не є в першій.

Варіант 24:

1. Словники: Дано словник з іменами та балами студентів. Збільште бал кожного студента на 10.
2. Множини: Напишіть програму, яка приймає множину чисел і виводить максимальне число множини.

Варіант 25:

1. Словники: Створіть словник, де ключами є числа від 1 до 5, а значеннями - їх факторіали.
2. Множини: Напишіть програму, яка приймає множину чисел і виводить мінімальне число множини.

Контрольні запитання

1. Що таке словник в Python?
2. Як створити словник в Python?
3. Що таке ключі та значення в словниках Python?
4. Як отримати значення зі словника за допомогою ключа?
5. Що відбудеться, якщо у словнику є два однакових ключа?
6. Що таке множина в Python?
7. Як створити множину в Python?
8. Як об'єднати дві множини в Python?
9. Що таке перетин множин?
10. Що таке об'єднання множин?
11. Що таке різниця множин?

ПРАКТИЧНА РОБОТА №8: ФАЙЛИ ТА МЕТОДИ РЯДКІВ У PYTHON

Мета роботи – навчитися роботі з файлами в Python, використовуючи базові операції зчитування і запису, а також опанувати роботу з методами рядків для обробки тексту.

Теоретичні відомості

Файли у Python.

Python надає вбудовані функції та методи для роботи з файлами. Ви можете виконувати різноманітні операції з файлами, такі як читання, запис, додавання вмісту, видалення та багато іншого.

Створення файлу. Для створення нового файлу в Python використовується функція `open()`. Якщо файл, який ви намагаєтеся відкрити, не існує, Python автоматично створить його.

```
file = open("file.txt", "w")  
file.close()
```

Запис в файл. Щоб записати вміст у файл, використовується метод `write()`.

```
file = open("file.txt", "w")  
file.write("Hello, world!")  
file.close()
```

Читання з файлу. Щоб прочитати вміст файлу, можна використати метод `read()`.

```
file = open("file.txt", "r")  
content = file.read()  
print(content)  
file.close()
```

Робота з рядками в Python.

В Python є різні методи для роботи з рядками. Ось декілька з них:

- `upper()`: Перетворює всі букви в рядку в верхній регістр.
- `replace(old, new)`: Замінює всі входження 'old' на 'new' в рядку.
- `count(substring)`: Підраховує кількість входжень 'substring' в рядку.

Приклад роботи:

```
s = "Hello, world!"
```

```
print(s.upper()) # Output: "HELLO, WORLD!"
```

```
print(s.replace("world", "Python")) # Output: "Hello, Python!"
```

```
print(s.count('o')) # Output: 2
```

Ці методи рядків можна використовувати для обробки вмісту файлу після його читання. Для цього потрібно прочитати вміст файлу в рядок, виконати необхідні операції з рядком, а потім записати отриманий результат назад в файл.

Практична частина

Варіант 1:

1. Створіть текстовий файл `file1.txt` і запишіть в нього рядок "Python is an interpreted high-level general-purpose programming language.". Після цього відкрийте цей файл, прочитайте його вміст і переведіть всі букви в верхній регістр. Результат запишіть у новий файл `file1_upper.txt`.
2. Створіть текстовий файл `file2.txt` і запишіть в нього рядок "Python's design philosophy emphasizes code readability with its notable use of significant indentation.". Після цього відкрийте цей файл, прочитайте його вміст і видаліть всі голосні букви. Результат запишіть у новий файл `file2_no_vowels.txt`.

Варіант 2:

1. Створіть текстовий файл file3.txt і запишіть в нього рядок "Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects.". Після цього відкрийте цей файл, прочитайте його вміст і переверніть порядок слів у реченні. Результат запишіть у новий файл file3_reversed.txt.
2. Створіть текстовий файл file4.txt і запишіть в нього рядок "Python is dynamically-typed and garbage-collected.". Після цього відкрийте цей файл, прочитайте його вміст і розділіть речення на окремі слова, кожне з яких записуйте в новий рядок. Результат запишіть у новий файл file4_words.txt.

Варіант 3:

1. Створіть текстовий файл file5.txt і запишіть в нього рядок "It supports multiple programming paradigms, including structured (particularly, procedural), object-oriented and functional programming.". Після цього відкрийте цей файл, прочитайте його вміст і підрахуйте кількість входжень кожного слова. Результат запишіть у новий файл file5_word_counts.txt.
2. Створіть текстовий файл file6.txt і запишіть в нього рядок "Python is often described as a 'batteries included' language due to its comprehensive standard library.". Після цього відкрийте цей файл, прочитайте його вміст і знайдіть довжину найдовшого слова. Результат запишіть у новий файл file6_longest_word.txt.

Варіант 4:

1. Створіть текстовий файл file7.txt і запишіть в нього рядок "Guido van Rossum began working on Python in the late 1980s, as a successor to the ABC language, and first released it in 1991 as Python 0.9.0". Після цього відкрийте цей файл, прочитайте його вміст і замініть всі входження "Python" на "PY". Результат запишіть у новий файл file7_replaced.txt.
2. Створіть текстовий файл file8.txt і запишіть в нього рядок "Python 2.0, released in 2000, introduced new features, such as list comprehensions and

a garbage collection system with reference counting, and was discontinued with version 2.7 in 2020". Після цього відкрийте цей файл, прочитайте його вміст і витягніть всі числа. Результат запишіть у новий файл file8_numbers.txt.

Варіант 5:

1. Створіть текстовий файл file9.txt і запишіть в нього рядок "Python 3.0, released in 2008, was a major revision of the language that is not completely backward-compatible, and much Python 2 code does not run unmodified on Python 3.". Після цього відкрийте цей файл, прочитайте його вміст і замініть всі голосні букви на "*". Результат запишіть у новий файл file9_vowels_replaced.txt.
2. Створіть текстовий файл file10.txt і запишіть в нього рядок "The Python 2 language was officially discontinued in 2020 (first planned for 2015), and 'Python 2.7.18 is the last Python 2.7 release and therefore the last Python 2 release.'". Після цього відкрийте цей файл, прочитайте його вміст і видаліть всі пробіли. Результат запишіть у новий файл file10_no_spaces.txt.

Варіант 6:

1. Створіть текстовий файл file11.txt і запишіть в нього рядок "Hello, World! This is Python.". Після цього відкрийте цей файл, прочитайте його вміст і переведіть всі букви в нижній регістр. Результат запишіть у новий файл file11_lower.txt.
2. Створіть текстовий файл file12.txt і запишіть в нього рядок "Python programming language is awesome.". Після цього відкрийте цей файл, прочитайте його вміст і переверніть порядок символів у рядку. Результат запишіть у новий файл file12_reversed.txt.

Варіант 7:

1. Створіть текстовий файл file13.txt і запишіть в нього рядок "Python is easy to learn and powerful.". Після цього відкрийте цей файл,

прочитайте його вміст і видаліть всі букви 'a'. Результат запишіть у новий файл file13_no_a.txt.

2. Створіть текстовий файл file14.txt і запишіть в нього рядок "Python is named after the comedy television show Monty Python's Flying Circus.". Після цього відкрийте цей файл, прочитайте його вміст і підрахуйте кількість слів у рядку. Результат запишіть у новий файл file14_word_count.txt.

Варіант 8:

1. Створіть текстовий файл file15.txt і запишіть в нього рядок "Python supports multiple programming paradigms.". Після цього відкрийте цей файл, прочитайте його вміст і замініть слово "Python" на "Py". Результат запишіть у новий файл file15_replaced.txt.
2. Створіть текстовий файл file16.txt і запишіть в нього рядок "Python is often used as a scripting language.". Після цього відкрийте цей файл, прочитайте його вміст і видаліть всі коми та крапки. Результат запишіть у новий файл file16_no_punctuation.txt.

Варіант 9:

1. Створіть текстовий файл file17.txt і запишіть в нього рядок "Python has a large standard library.". Після цього відкрийте цей файл, прочитайте його вміст і видаліть всі пробіли. Результат запишіть у новий файл file17_no_spaces.txt.
2. Створіть текстовий файл file18.txt і запишіть в нього рядок "Python is dynamically typed and garbage-collected.". Після цього відкрийте цей файл, прочитайте його вміст і витягніть всі числа. Результат запишіть у новий файл file18_numbers.txt.

Варіант 10:

1. Створіть текстовий файл file19.txt і запишіть в нього рядок "Python was conceived in the late 1980s.". Після цього відкрийте цей файл, прочитайте його вміст і витягніть всі числа. Результат запишіть у новий файл file19_numbers.txt.

2. Створіть текстовий файл file20.txt і запишіть в нього рядок "Python was created by Guido van Rossum.". Після цього відкрийте цей файл, прочитайте його вміст і знайдіть довжину найдовшого слова. Результат запишіть у новий файл file20_longest_word.txt.

Варіант 11:

1. Створіть текстовий файл file21.txt і запишіть в нього рядок "Python is great for backend web development.". Після цього відкрийте цей файл, прочитайте його вміст і перетворіть всі голосні на прописні. Результат запишіть у новий файл file21_capital_vowels.txt.
2. Створіть текстовий файл file22.txt і запишіть в нього рядок "Python is a high-level general-purpose programming language.". Після цього відкрийте цей файл, прочитайте його вміст і видаліть всі букви 'p' та 'P'. Результат запишіть у новий файл file22_no_p.txt.

Варіант 12:

1. Створіть текстовий файл file23.txt і запишіть в нього рядок "Python is highly readable.". Після цього відкрийте цей файл, прочитайте його вміст і переверніть порядок слів у рядку. Результат запишіть у новий файл file23_reversed.txt.
2. Створіть текстовий файл file24.txt і запишіть в нього рядок "Python is popular among developers.". Після цього відкрийте цей файл, прочитайте його вміст і замініть всі пробіли на дефіси. Результат запишіть у новий файл file24_hyphens.txt.

Варіант 13:

1. Створіть текстовий файл file25.txt і запишіть в нього рядок "Python is widely used in scientific and numeric computing.". Після цього відкрийте цей файл, прочитайте його вміст і підрахуйте кількість букв 'i' в рядку. Результат запишіть у новий файл file25_i_count.txt.
2. Створіть текстовий файл file26.txt і запишіть в нього рядок "Python is designed to be easy to understand and fun to use.". Після цього відкрийте

цей файл, прочитайте його вміст і видаліть всі букви 'e'. Результат запишіть у новий файл file26_no_e.txt.

Варіант 14:

1. Створіть текстовий файл file27.txt і запишіть в нього рядок "Python has an active and friendly community.". Після цього відкрийте цей файл, прочитайте його вміст і знайдіть довжину найдовшого слова в рядку. Результат запишіть у новий файл file27_longest_word.txt.
2. Створіть текстовий файл file28.txt і запишіть в нього рядок "Python is an interpreted, high-level and general-purpose programming language.". Після цього відкрийте цей файл, прочитайте його вміст і замініть всі коми на крапки. Результат запишіть у новий файл file28_commas_to_dots.txt.

Варіант 15:

1. Створіть текстовий файл file29.txt і запишіть в нього рядок "Python supports multiple programming paradigms.". Після цього відкрийте цей файл, прочитайте його вміст і знайдіть слово з найменшою кількістю букв. Результат запишіть у новий файл file29_shortest_word.txt.
2. Створіть текстовий файл file30.txt і запишіть в нього рядок "Python features a dynamic type system and automatic memory management.". Після цього відкрийте цей файл, прочитайте його вміст і замініть всі голосні на числа від 0 до 4. Результат запишіть у новий файл file30_vowels_to_numbers.txt.

Варіант 16:

1. Створіть текстовий файл file31.txt і запишіть в нього рядок "Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance.". Після цього відкрийте цей файл, прочитайте його вміст і замініть всі пробіли на нижнє підкреслення. Результат запишіть у новий файл file31_spaces_to_underscore.txt.
2. Створіть текстовий файл file32.txt і запишіть в нього рядок "Python supports modules and packages, which encourages program modularity and

code reuse.". Після цього відкрийте цей файл, прочитайте його вміст і підрахуйте кількість букв 'o' в рядку. Результат запишіть у новий файл file32_o_count.txt.

Варіант 17:

1. Створіть текстовий файл file33.txt і запишіть в нього рядок "The Python interpreter and the extensive standard library are available in source or binary form without charge for all major platforms.". Після цього відкрийте цей файл, прочитайте його вміст і перетворіть всі букви 'a' в рядку на прописні. Результат запишіть у новий файл file33_a_to_uppercase.txt.
2. Створіть текстовий файл file34.txt і запишіть в нього рядок "Python interpreters are available for many operating systems.". Після цього відкрийте цей файл, прочитайте його вміст і знайдіть найдовше слово в рядку. Результат запишіть у новий файл file34_longest_word.txt.

Варіант 18:

1. Створіть текстовий файл file35.txt і запишіть в нього рядок "Python is a multi-paradigm programming language.". Після цього відкрийте цей файл, прочитайте його вміст і замініть всі коми на крапки. Результат запишіть у новий файл file35_commas_to_dots.txt.
2. Створіть текстовий файл file36.txt і запишіть в нього рядок "Python is dynamically typed and garbage-collected.". Після цього відкрийте цей файл, прочитайте його вміст і замініть всі голосні на числа від 0 до 4. Результат запишіть у новий файл file36_vowels_to_numbers.txt.

Варіант 19:

1. Створіть текстовий файл file37.txt і запишіть в нього рядок "Python supports object-oriented, imperative, and functional or procedural programming styles.". Після цього відкрийте цей файл, прочитайте його вміст і замініть всі пробіли на нижнє підкреслення. Результат запишіть у новий файл file37_spaces_to_underscore.txt.

2. Створіть текстовий файл file38.txt і запишіть в нього рядок "Python interpreters are available for many operating systems.". Після цього відкрийте цей файл, прочитайте його вміст і замініть всі коми на крапки. Результат запишіть у новий файл file38_commas_to_dots.txt.

Варіант 20:

1. Створіть текстовий файл file39.txt і запишіть в нього рядок "Python is widely used in artificial intelligence, natural language generation, neural networks, and other advanced fields of computer science.". Після цього відкрийте цей файл, прочитайте його вміст і замініть всі пробіли на нижнє підкреслення. Результат запишіть у новий файл file39_spaces_to_underscore.txt.
2. Створіть текстовий файл file40.txt і запишіть в нього рядок "Python was conceived in the late 1980s as a successor to the ABC language.". Після цього відкрийте цей файл, прочитайте його вміст і підрахуйте кількість букв 'o' в рядку. Результат запишіть у новий файл file40_o_count.txt.

Варіант 21:

1. Створіть текстовий файл file41.txt і запишіть в нього рядок "Python is an interpreted high-level programming language for general-purpose programming.". Після цього відкрийте цей файл, прочитайте його вміст і замініть всі букви 'p' на 'P'. Результат запишіть у новий файл file41_p_to_P.txt.
2. Створіть текстовий файл file42.txt і запишіть в нього рядок "Python has a design philosophy which emphasizes code readability.". Після цього відкрийте цей файл, прочитайте його вміст і підрахуйте кількість слів в рядку. Результат запишіть у новий файл file42_word_count.txt.

Варіант 22:

1. Створіть текстовий файл file43.txt і запишіть в нього рядок "Python's syntax allows programmers to express concepts in fewer lines of code.". Після цього відкрийте цей файл, прочитайте його вміст і знайдіть

найдовше слово в рядку. Результат запишіть у новий файл file43_longest_word.txt.

2. Створіть текстовий файл file44.txt і запишіть в нього рядок "Python is a popular language for web development.". Після цього відкрийте цей файл, прочитайте його вміст і перетворіть всі букви 'a' в рядку на прописні. Результат запишіть у новий файл file44_a_to_uppercase.txt.

Варіант 23:

1. Створіть текстовий файл file45.txt і запишіть в нього рядок "Python's large standard library, commonly cited as one of its greatest strengths.". Після цього відкрийте цей файл, прочитайте його вміст і підрахуйте кількість букв 'o' в рядку. Результат запишіть у новий файл file45_o_count.txt.
2. Створіть текстовий файл file46.txt і запишіть в нього рядок "Python was conceived in the late 1980s as a successor to the ABC language.". Після цього відкрийте цей файл, прочитайте його вміст і замініть всі коми на крапки. Результат запишіть у новий файл file46_commas_to_dots.txt.

Варіант 24:

1. Створіть текстовий файл file47.txt і запишіть в нього рядок "Python is an interpreted high-level programming language for general-purpose programming.". Після цього відкрийте цей файл, прочитайте його вміст і підрахуйте кількість букв 'n' в рядку. Результат запишіть у новий файл file47_n_count.txt.
2. Створіть текстовий файл file48.txt і запишіть в нього рядок "Python has a design philosophy which emphasizes code readability.". Після цього відкрийте цей файл, прочитайте його вміст і підрахуйте кількість голосних в рядку. Результат запишіть у новий файл file48_vowel_count.txt.

Варіант 25:

1. Створіть текстовий файл file49.txt і запишіть в нього рядок "Python's syntax allows programmers to express concepts in fewer lines of code.". Після цього відкрийте цей файл, прочитайте його вміст і підрахуйте кількість пробілів в рядку. Результат запишіть у новий файл file49_space_count.txt.
2. Створіть текстовий файл file50.txt і запишіть в нього рядок "Python is a popular language for web development.". Після цього відкрийте цей файл, прочитайте його вміст і перетворіть всі букви 'w' в рядку на прописні. Результат запишіть у новий файл file50_w_to_uppercase.txt.

Контрольні запитання

1. Як створити файл у Python?
2. Що таке режими файлу і які з них доступні в Python?
3. Які методи використовуються для запису та читання файлів в Python?
4. Як закрити файл в Python і чому це важливо?
5. Що таке контекстний менеджер в Python і як його використовувати при роботі з файлами?

ПРАКТИЧНА РОБОТА №9: СТВОРЕННЯ ГРАФІЧНОГО ІНТЕРФЕЙСУ У PYTHON

Мета роботи – опанувати роботу модулю Tkinter у Python для створення графічного інтерфейсу для вирішення практичних завдань.

Теоретичні відомості

Tkinter – це стандартна бібліотека Python для створення графічного інтерфейсу користувача (GUI). Вона входить у стандартний пакет Python, тому її не потрібно встановлювати окремо.

Основні компоненти Tkinter:

- Вікно (window): Головний контейнер, в якому розміщуються всі інші віджети.
- Віджети (widgets): Елементи, які розміщуються в вікні. Це можуть бути кнопки, текстові поля, мітки, меню тощо.

Створення вікна в Tkinter.

```
from tkinter import Tk  
root = Tk()  
root.mainloop()
```

Цей код створює порожнє вікно.

Додавання віджетів.

- Текстове поле:

```
from tkinter import Tk, Entry  
root = Tk()  
e = Entry(root)  
e.pack()  
root.mainloop()
```

Цей код створює вікно з текстовим полем.

- Кнопки:

```
from tkinter import Tk, Button  
root = Tk()  
button = Button(root, text="Натисніть мене")  
button.pack()  
root.mainloop()
```

Текст на кнопці задається параметром text.

Обробка подій.

Для обробки подій (наприклад, натискання кнопки) використовуються функції-обробники. Нижче приклад створення кнопки, яка при натисканні друкує текст в консоль:

```
from tkinter import Tk, Button  
def handle_click():  
    print("Кнопка була натиснута!")  
root = Tk()  
button = Button(root, text="Натисніть мене", command=handle_click)  
button.pack()  
root.mainloop()
```

Коли користувач натискає кнопку, викликається функція handle_click(), яка друкує текст в консоль. За допомогою цих основних принципів можна створювати складніші інтерфейси з різними віджетами і обробниками подій.

Канвас в Tkinter.

Канвас (Canvas) – це віджет в Tkinter, що дозволяє робити малюнки, такі як лінії, прямокутники, овали та інші геометричні фігури. Також на ньому можна розміщувати інші віджети.

Створення канвасу в Tkinter:

```
from tkinter import Tk, Canvas  
root = Tk()  
canvas = Canvas(root, width=400, height=300)  
canvas.pack()  
root.mainloop()
```

Цей код створює вікно з канвасом розміром 400x300 пікселів.

Геометричні фігури в Tkinter.

На канвасі можна створювати різні геометричні фігури:

- Лінії: Щоб створити лінію, використовується метод `create_line()`.

Наприклад, `canvas.create_line(10, 10, 200, 50)` створить лінію від точки (10,10) до точки (200,50).

- Прямокутники: Метод `create_rectangle()` використовується для створення прямокутників. Наприклад, `canvas.create_rectangle(10, 10, 200, 50)` створить прямокутник, верхній лівий кут якого знаходиться в точці (10,10), а нижній правий - в точці (200,50).

- Овали: Метод `create_oval()` створює овал в межах заданого прямокутника. Наприклад, `canvas.create_oval(10, 10, 200, 50)` створить овал, що вписаний в прямокутник з верхнім лівим кутом (10,10) і нижнім правим кутом (200,50).

- Полігони: Метод `create_polygon()` створює полігон з вказаними вершинами. Наприклад, `canvas.create_polygon(10, 10, 200, 50, 100, 100)` створить трикутник з вершинами в точках (10,10), (200,50) та (100,100).

Відкриття файлів в Tkinter.

Для відкриття файлів можна використати діалогове вікно відкриття файлу:

```
from tkinter import Tk, filedialog  
root = Tk()  
filename = filedialog.askopenfilename()
```

`root.mainloop()`

Практична частина

Зауваження: у всіх варіантах завдань, навіть якщо не вказано прямо, необхідно розробити простий графічний інтерфейс, який буде емулювати роботи програми, а саме поля для вводу-вивода інформації, кнопки перетворень, та інше.

Варіант 1:

Створити простий графічний інтерфейс за допомогою Tkinter, який містить два текстових поля і одну кнопку. При натисканні на кнопку програма повинна брати значення з першого текстового поля, виконувати над ним якусь операцію (наприклад, обертати рядок або додавати до нього певний текст), і виводити результат у друге текстове поле.

Варіант 2:

Створити програму, яка зчитує введене число з текстового поля і малює таку кількість квадратів на канвасі Tkinter. Додати кнопку, яка при натисканні очищує канвас (`Canvas, canvas.create_rectangle(x1, y1, x2, y2, fill = fill_color)`).

Варіант 3:

Створити простий графічний калькулятор за допомогою Tkinter. Інтерфейс повинен містити кнопки для вводу чисел, основних математичних операцій (додавання, віднімання, множення, ділення), а також текстове поле для відображення результату.

Варіант 4:

Створити простий текстовий редактор. Інтерфейс повинен містити велике текстове поле для введення і редагування тексту, а також кнопки "Зберегти" і "Відкрити", які дозволяють зберегти поточний текст в файл або відкрити існуючий текстовий файл.

Варіант 5:

Створити простий інтерфейс та програму, яка відтворює просту гру "Камінь, ножиці, папір". Користувач може вибирати одну з трьох опцій, а програма вибирає випадковим чином. Потім виводиться результат.

Варіант 6:

Створити простий інтерфейс та програму-записник, яка дозволяє користувачеві зберігати короткі нотатки.

Варіант 7:

Створити простий інтерфейс та просту програму для перетворення валют. Інтерфейс має включати поля для введення кількості валюти, випадаючі списки для вибору валют, а також кнопку для виконання перетворення.

Варіант 8:

Створити простий інтерфейс та програму, яка демонструє різні геометричні фігури на канвасі Tkinter. Користувач може вибирати фігуру з випадаючого списку (Canvas, canvas.create_rectangle, canvas.create_oval, тощо).

Варіант 9:

Створити простий інтерфейс та програму, яка дозволяє користувачеві вводити текст і підраховує кількість слів у тексті. Програма повинна містити текстові поля для вводу\виводу інформації та кнопку виконання дії.

Варіант 10:

Створити простий інтерфейс та програму, яка відтворює простий квіз. Користувачу пропонуються питання з декількома варіантами відповіді, а програма підраховує кількість правильних відповідей.

Варіант 11:

Розробити простий інтерфейс та програму, яка перетворює між різними системами мір (температура, відстань, вага тощо). Програма повинна містити текстові поля для вводу\виводу інформації та кнопку виконання дії.

Варіант 12:

Створити простий інтерфейс та програму, яка виконує прості математичні операції з комплексними числами. Програма повинна містити текстові поля для вводу\виводу інформації та кнопку виконання дії.

Варіант 13:

Розробити простий інтерфейс та програму, яка виконує аналіз тексту: визначає кількість слів, речень, унікальних слів та інше. Програма повинна містити текстові поля для вводу\виводу інформації та кнопку виконання дії.

Варіант 14:

Розробити простий інтерфейс та програму, яка виконує прості геометричні обчислення (обчислення площі, периметра, об'єму різних геометричних фігур).

Варіант 15:

Розробити простий інтерфейс та програму, яка перетворює введений текст в верхній регістр. Програма повинна містити текстові поля для вводу\виводу інформації та кнопку виконання дії.

Варіант 16:

Створити простий інтерфейс та програму, яка обчислює кількість букв в введеному рядку. Програма повинна містити текстові поля для вводу\виводу інформації та кнопку виконання дії.

Варіант 17:

Розробити простий інтерфейс та програму, яка перетворює введений текст в нижній регістр. Програма повинна містити текстові поля для вводу\виводу інформації та кнопку виконання дії.

Варіант 18:

Розробити простий інтерфейс та програму, яка обчислює довжину введеного тексту. Програма повинна містити текстові поля для вводу\виводу інформації та кнопку виконання дії.

Варіант 19:

Розробити простий інтерфейс з текстовим полем для вводу та кнопкою "Перевести в верхній регістр". При натисканні на кнопку програма повинна перетворити введений текст в верхній регістр і вивести результат у новому вікні.

Варіант 20:

Створити простий графічний інтерфейс, який містить два текстових поля та кнопку. При натисканні на кнопку програма повинна брати значення з першого текстового поля (припустимо, це число), піднести це число до квадрату і вивести результат у друге текстове поле.

Варіант 21:

Створити програму, яка зчитує введене число з текстового поля і малює на канвасі Tkinter круг з відповідним радіусом. Додати кнопку, яка при натисканні очищує канвас.

Варіант 22:

Створити простий графічний інтерфейс, який містить два текстових поля та кнопку. При натисканні на кнопку програма повинна брати значення з першого текстового поля, перетворити його на список слів (розділити по пробілам), і вивести цей список у друге текстове поле.

Варіант 23:

Створити простий графічний інтерфейс, який містить два текстових поля та кнопку. При натисканні на кнопку програма повинна брати значення з першого текстового поля, замінити всі голосні букви на "а", і вивести результат у друге текстове поле.

Варіант 24:

Створити програму, яка зчитує введене число з текстового поля і малює на канвасі Tkinter відповідну кількість кольорних кругів різного діаметру випадково розташованих на канвасі. Додати кнопку, яка при натисканні очищує канвас.

Варіант 25:

Створити програму, яка зчитує введене число з текстового поля і малює на канвасі Tkinter відповідну кількість випадкових точок. Додати кнопку, яка при натисканні очищує канвас.

Контрольні запитання

1. Що таке Tkinter? Яке його призначення в Python?

2. Які основні компоненти в Tkinter? Опишіть їх.
3. Як створити вікно в Tkinter? Наведіть приклад коду.
4. Як додати віджети, такі як кнопки та текстові поля, до вікна Tkinter?
Наведіть приклад коду.
5. Що таке Canvas в Tkinter і для чого він використовується?

ПРАКТИЧНА РОБОТА №10: ООП. КЛАСИ У PYTHON

Мета роботи – засвоїти основи об'єктно-орієнтовного програмування та особливостей реалізації класів у Python.

Теоретичні відомості

Об'єктно-орієнтоване програмування (ООП) є методологією програмування, яка організує програмний код за допомогою "об'єктів" – структур даних, що складаються з поля даних та методів для їх обробки. Основні поняття ООП в Python включають класи, об'єкти (екземпляри класу), атрибути і методи.

Класи і об'єкти.

Клас визначає структуру об'єкту і є своєрідним "шаблоном", згідно з яким створюються об'єкти.

```
class MyClass:
    x = 5

p1 = MyClass() # створюємо об'єкт p1
print(p1.x) # виводимо атрибут x об'єкта p1
```

Атрибути і методи.

Атрибути – це властивості, які має клас або його екземпляр. Методи – це функції, визначені в класі, які можуть маніпулювати атрибутами класу або екземпляра.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def bark(self):
        print("Woof!")
```

```
d = Dog("Fido", 3)
d.bark() # виводить "Woof!"
```

`__init__` метод.

`__init__` є спеціальним методом, який викликається при створенні об'єкта. Він використовується для ініціалізації атрибутів об'єкта.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Наслідування.

Наслідування – це можливість створювати новий клас на основі вже існуючого. Новий клас наслідує всі атрибути і методи батьківського класу і може мати додаткові.

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
    def eat(self):
        print("Nom nom nom")
```

```
class Dog(Animal):
    def __init__(self, name, age, breed):
        super().__init__(name, age) # виклик __init__ методу
        # батьківського класу
        self.breed = breed # додатковий атрибут для класу Dog

    def bark(self):
```

```
print("Woof!")
```

```
d = Dog("Fido", 3, "Bulldog")  
d.eat() # виводить "Nom nom nom"  
d.bark() # виводить "Woof!"
```

У цьому прикладі Dog є підкласом Animal і наслідує його атрибути та методи. Ми також додаємо додатковий атрибут breed та метод bark() специфічні для Dog.

Слово super використовується для виклику методів батьківського класу. У цьому випадку, ми викликаємо __init__ метод Animal для ініціалізації атрибутів name та age.

Поліморфізм.

Поліморфізм в ООП – це здатність об'єктів різних типів виконувати однакові дії різними способами. Зазвичай, це досягається за допомогою використання наслідування та перевизначення методів.

```
class Cat(Animal):  
    def __init__(self, name, age, breed):  
        super().__init__(name, age)  
        self.breed = breed  
    def make_sound(self):  
        print("Meow!")  
class Dog(Animal):  
    def __init__(self, name, age, breed):  
        super().__init__(name, age)  
        self.breed = breed  
    def make_sound(self):  
        print("Woof!")
```

```
c = Cat("Whiskers", 3, "Siamese")
```

```
d = Dog("Fido", 3, "Bulldog")
```

```
animals = [c, d]
```

```
for animal in animals:
```

```
    animal.make_sound() # Cat виводить "Meow!", Dog виводить  
    "Woof!"
```

У цьому прикладі, обидва Cat та Dog мають метод `make_sound()`, але вони реалізовані по-різному. Це є прикладом поліморфізму.

Практична частина

Зауваження: всі класи, що створюються, повинні обов'язково містити конструктор та деструктор класу.

Варіант 1:

1. Створити клас `Car` з атрибутами: `make`, `model`, `year`. Додайте метод `get_description()`, який повертає рядок з повною інформацією про автомобіль.
2. Розширити клас `Car` новим атрибутом `odometer_reading` та методом `read_odometer()`, який виводить поточний пробіг автомобіля.

Варіант 2:

1. Створити клас `Dog` з атрибутами `name` і `age` та методом `bark()`, який друкує "Woof!".
2. Розширити клас `Dog` методом `get_human_age()`, який перетворює вік собаки на "людські" роки (1 рік собаки = 7 людських років).

Варіант 3:

1. Створити клас `Circle` з атрибутом `radius` та методом `area()`, який обчислює площу круга.
2. Додати до класу `Circle` метод `perimeter()`, який обчислює периметр круга.

Варіант 4:

1. Створити клас BankAccount з атрибутами owner, balance та методами deposit(), withdraw().
2. Розширити клас BankAccount методом transfer(), який реалізує переказ коштів з одного рахунку на інший.

Варіант 5:

1. Створити клас Student з атрибутами name, age, grades (список оцінок) та методом average_grade(), який обчислює середній бал студента.
2. Розширити клас Student методом add_grade(), який додає нову оцінку до списку оцінок.

Варіант 6:

1. Створити клас Book з атрибутами title, author, publishing_year та методом get_book_info(), який виводить інформацію про книгу.
2. Розширити клас Book методом is_new(), який визначає, чи є книга новою (наприклад, видана не більше ніж 3 роки тому).

Варіант 7:

1. Створити клас Employee з атрибутами name, salary та методом raise_salary(), який збільшує зарплату на вказану суму.
2. Розширити клас Employee методом years_to_retirement(), який визначає, скільки років залишилось до пенсії (наприклад, припустимо, що вік співробітника 60 років).

Варіант 8:

1. Створити клас Rectangle з атрибутами length і width та методом area(), який обчислює площу прямокутника.
2. Розширити клас Rectangle методом perimeter(), який обчислює периметр прямокутника.

Варіант 9:

1. Створити клас Movie з атрибутами title, director, release_year та методом get_movie_info(), який виводить інформацію про фільм.
2. Розширити клас Movie методом is_old(), який визначає, чи є фільм старим (наприклад, випущено більше ніж 20 років тому).

Варіант 10:

1. Створити клас Calculator з методами add(), subtract(), multiply(), divide().
2. Додати до класу Calculator метод power(), який підносить одне число до степені іншого числа.

Варіант 11:

1. Створити клас Vector з атрибутами x, y і z. Додати метод для обчислення довжини вектора.
2. Додати методи до класу Vector для обчислення скалярного і векторного добутків двох векторів.

Варіант 12:

1. Створити клас Box з атрибутами height, width, depth. Додати метод для обчислення об'єму коробки.
2. Додати метод до класу Box, що обчислює, чи може одна коробка вміститися в іншу.

Варіант 13:

1. Створити клас ComplexNumber з атрибутами real, imaginary. Додати метод для виводу комплексного числа у вигляді рядка.
2. Додати методи до класу ComplexNumber для обчислення суми, різниці, добутку і частки двох комплексних чисел.

Варіант 14:

1. Створити клас Student з атрибутами name, age, subjects (словник з предметами і оцінками). Додати метод для обчислення середнього балу студента.
2. Додати метод до класу Student для додавання нового предмету і оцінки до словника subjects.

Варіант 15:

1. Створити клас Person з атрибутами name, age, address. Додати метод для виводу всієї інформації про персону.

2. Додати метод до класу `Person`, що визначає, чи є ця людина повнолітньою (вік 18 або більше).

Варіант 16:

1. Створити клас `Product` з атрибутами `name`, `price`, `quantity`. Додати метод для обчислення загальної вартості всіх одиниць продукту.
2. Додати метод до класу `Product`, який зменшує кількість товару на задану величину.

Варіант 17:

1. Створити клас `Triangle` з атрибутами `a`, `b`, `c` (довжини сторін). Додати метод для обчислення периметра трикутника.
2. Додати метод до класу `Triangle` для обчислення площі трикутника за формулою Герона.

Варіант 18:

1. Створити клас `Animal` з атрибутами `name`, `age`, `weight`. Додати метод для виводу всієї інформації про тварину.
2. Додати метод до класу `Animal` для збільшення ваги тварини на задану величину.

Варіант 19:

1. Створити клас `Fraction` з атрибутами `numerator`, `denominator`. Додати метод для виводу дробу у вигляді рядка.
2. Додати методи до класу `Fraction` для обчислення суми, різниці, добутку і частки двох дробів.

Варіант 20:

1. Створити клас `Matrix` з атрибутом `matrix` (двовимірний список). Додати метод для виводу матриці.
2. Додати метод до класу `Matrix` для обчислення суми двох матриць.

Варіант 21:

1. Створити клас `Point` з атрибутами `x` і `y`. Додати метод для виводу точки у вигляді рядка.

2. Додати метод до класу Point для обчислення відстані між двома точками.

Варіант 22:

1. Створити клас Phone з атрибутами brand, model, price. Додати метод для виводу інформації про телефон.
2. Додати метод до класу Phone для зменшення ціни телефону на заданий відсоток.

Варіант 23:

1. Створити клас Polygon з атрибутом sides (список з довжинами сторін). Додати метод для обчислення периметра багатокутника.
2. Додати метод до класу Polygon для додавання нової сторони до списку sides.

Варіант 24:

1. Створити клас Ticket з атрибутами destination, price. Додати метод для виводу інформації про квиток.
2. Додати метод до класу Ticket для збільшення ціни квитка на заданий відсоток.

Варіант 25:

1. Створити клас Cube з атрибутом side_length. Додати метод для обчислення об'єму куба.
2. Додати метод до класу Cube для обчислення площі поверхні куба.

Контрольні запитання

1. Що таке об'єктно-орієнтоване програмування?
2. Що таке клас в контексті ООП в Python?
3. Що таке об'єкт (або екземпляр класу) в Python?
4. Що таке атрибути в Python?
5. Що таке методи в Python?

ПРАКТИЧНА РОБОТА №11: РЕГУЛЯРНІ ВИРАЗИ У PYTHON

Мета роботи – набуття практичних навичок в роботі з рядками в Python з використанням бібліотеки регулярних виразів.

Теоретичні відомості

Регулярні вирази, або *regex*, – це секвенції символів, що формують шаблон пошуку. Ці шаблони потім використовуються в функціях пошуку та заміни тексту.

Python має вбудований модуль для роботи з регулярними виразами, називається *re*. Для використання регулярних виразів вам потрібно імпортувати модуль за допомогою команди `import re`.

Ось кілька основних функцій, які надає бібліотека *re*:

- `re.match()` : Ця функція намагається зіставити регулярний вираз з початку рядка. Якщо збігів немає, вона повертає `None`.
- `re.search()` : Ця функція працює так само, як `match()`, але вона шукає збіг в усьому рядку - не тільки на початку.
- `re.findall()` : Виявляє всі збіги з регулярним виразом в рядку і повертає їх у вигляді списку рядків. Кожен повернений рядок представляє один збіг.
- `re.split()` : Ця функція розбиває рядок на підрядки, розділені регулярним виразом.
- `re.sub()` : Замінює одну або багато збігів регулярного виразу на рядок.

Приклад використання кількох з цих функцій:

```
import re
```

```
str = "The rain in Spain"
```

```
x = re.findall("ai", str)
```

```
print(x) # Output: ['ai', 'ai']
```

```
y = re.split("\s", str)
print(y) # Output: ['The', 'rain', 'in', 'Spain']
z = re.sub("\s", "_", str)
print(z) # Output: 'The_rain_in_Spain'
```

Крім того, ре використовує ряд спеціальних символів, таких як:

- `\d` : будь-яке число (розуміється як [0-9])
- `\D` : будь-який символ, крім цифри
- `\s` : будь-який пробіл
- `\S` : будь-який символ, крім пробілу
- `\w` : будь-яка буква, цифра або знак підкреслення (розуміється як [a-zA-Z0-9_])
- `\W` : будь-який символ, крім букви, цифри або знака підкреслення
- `.` : будь-який символ, крім нового рядка

Важливо: щоб використовувати ці спеціальні символи, ви повинні ставити `r` перед рядком регулярного виразу, щоб Python розумів його як "сирий" рядок. Наприклад, `r"\d"` визначає регулярний вираз, який шукає всі цифри в рядку.

Практична частина

Зауваження: Всі завдання виконувати за допомогою бібліотеки regular expression Python (*pip install regex*).

Варіант 1:

1. Рядок: "Hello world! Python is a great language for beginners. Number 1 choice of 2023.". Розбийте цей рядок на список слів.
2. Рядок: "Artificial intelligence (AI) is intelligence demonstrated by machines. Phone: 123-456-7890". Оберніть порядок слів у рядку.

Варіант 2:

1. Рядок: "The quick brown fox jumps over the lazy dog. My e-mail: example@example.com". Видаліть всі голосні букви з рядка.
2. Рядок: "Weather today: sunny, 25°C. Perfect for a walk!". Замініть всі пробіли на символ "_".

Варіант 3:

1. Рядок: "The total amount is 20.75 and the change is 5.25". Знайдіть всі числа у цьому рядку.
2. Рядок: "Today Is A Beautiful Day". Замініть всі великі букви на малі.

Варіант 4:

1. Рядок: "Artificial Intelligence is the future". Підрахуйте кількість входжень кожної букви у рядку.
2. Рядок: "Python is easy to learn, easy to read, and powerful". Підрахуйте кількість слів у рядку.

Варіант 5:

1. Рядок: "Researchers are developing and promoting friendly AI for the benefit of all humanity". Знайдіть довжину найкоротшого слова у рядку.
2. Рядок: "Python, from multi-paradigm programming languages, is great!". Знайдіть довжину найдовшого слова у рядку.

Варіант 6:

1. Рядок: "Artificial_Intelligence: the ability of a digital machine to perform tasks commonly associated with intelligent beings". Замініть усі символи "_", на пробіл.
2. Рядок: "Strengths of Python include its simplicity, readability, and extensive standard library". Оберніть порядок символів у рядку.

Варіант 7:

1. Рядок: "In 2023, ANDII was released by Google, and it's quite an achievement". Видаліть всі числа з рядка.
2. Рядок: "Sunrise@05:30am, Sunset@07:45pm". Витягніть час сходу та заходу сонця з рядка.

Варіант 8:

1. Рядок: "Human language processing is the main focus of AI". Підрахуйте кількість голосних букв у рядку.
2. Рядок: "To make a great project, teamwork is very essential". Підрахуйте кількість приголосних букв у рядку.

Варіант 9:

1. Рядок: "Technology is advancing at a rapid pace, and AI is at the forefront". Знайдіть всі слова, що починаються на "a".
2. Рядок: "This string contains special characters like @, #, &, and *". Знайдіть всі спеціальні символи у рядку.

Варіант 10:

1. Рядок: "Python is powerful... and fast; plays well with others; runs everywhere; is friendly & easy to learn; is Open.". Знайдіть всі розділові знаки в рядку.
2. Рядок: "Computer science is the study of algorithmic processes, computational machines, and computation itself.". Розбийте цей рядок на список слів.

Варіант 11:

1. Рядок: "To err is human, but to really foul things up you need a computer.". Замініть усі коми на точки.
2. Рядок: "SEMANTIC is a large language model trained by VSC.". Видаліть все, крім назв " SEMANTIC " та " VSC".

Варіант 12:

1. Рядок: "In 2023, GIA released ISA, it's a major achievement in AI.". Витягніть всі числа з рядка.
2. Рядок: "Life would be much easier if I had the source code.". Підрахуйте кількість слів у рядку.

Варіант 13:

1. Рядок: "Python - High-level, interpreted, interactive and object-oriented scripting language.". Видаліть усі знаки "-", та розділіть рядок на слова.

2. Рядок: "The only way to learn a new programming language is by writing programs in it.". Перетворіть рядок так, щоб всі слова були написані з великої літери.

Варіант 14:

1. Рядок: "No one has been able to create a machine that can duplicate the creativity of the human mind.". Знайдіть довжину найдовшого та найкоротшого слова у рядку.
2. Рядок: "Artificial Intelligence is making machines intelligent, making them capable of thinking and learning.". Видаліть всі голосні букви з рядка.

Варіант 15:

1. Рядок: "The internet is becoming the town square for the global village of tomorrow.". Знайдіть всі слова, що закінчуються на "e".
2. Рядок: "Python can be easy to pick up whether you're a first time programmer or you're experienced with other languages.". Виведіть усі слова у алфавітному порядку.

Варіант 16:

1. Рядок: "I do not fear computers. I fear lack of them. - Isaac Asimov". Підрахуйте кількість пробілів у рядку.
2. Рядок: "The most technologically efficient machine that man has ever invented is the book. - Northrop Frye". Видаліть всі букви "e" з рядка.

Варіант 17:

1. Рядок: "The science of today is the technology of tomorrow. - Edward Teller". Замініть всі букви "t" на "T".
2. Рядок: "If we continue to develop our technology without wisdom or prudence, our servant may prove to be our executioner. - Omar N. Bradley". Замініть всі входження слова "our" на "OUR".

Варіант 18:

1. Рядок: "Science and technology revolutionize our lives, but memory, tradition and myth frame our response. - Arthur M. Schlesinger". Замініть всі великі букви на малі.
2. Рядок: "Technology... the knack of so arranging the world that we don't have to experience it. - Max Frisch". Замініть всі малі букви на великі.

Варіант 19:

1. Рядок: "The great myth of our times is that technology is communication. - Libby Larsen". Перевірте, чи рядок починається на "The".
2. Рядок: "We are stuck with technology when what we really want is just stuff that works. - Douglas Adams". Перевірте, чи рядок закінчується на "Adams".

Варіант 20:

1. Рядок: "Our technology forces us to live mythically, but we continue to think fragmentarily, and on single, separate, and mechanical levels. - Marshall McLuhan". Розділіть рядок по пробілам та виведіть кожне слово на новому рядку.
2. Рядок: "We've arranged a civilization in which most crucial elements profoundly depend on science and technology. - Carl Sagan". Знайдіть кількість входжень слова "technology".

Варіант 21:

1. Рядок: "Technology is just a tool. In terms of getting the kids working together and motivating them, the teacher is the most important. - Bill Gates". Знайдіть кількість чисел у рядку.
2. Рядок: "Technology is nothing. What's important is that you have a faith in people, that they're basically good and smart, and if you give them tools, they'll do wonderful things with them. - Steve Jobs". Замініть всі входження слова "technology" на "TECHNOLOGY".

Варіант 22:

1. Рядок: "Any sufficiently advanced technology is indistinguishable from magic. - Arthur C. Clarke". Перевірте, чи рядок починається на "Any".

2. Рядок: "Science and technology multiply around us. To an increasing extent they dictate the languages in which we speak and think. Either we use those languages, or we remain mute. - J. G. Ballard". Перевірте, чи рядок закінчується на "Ballard".

Варіант 23:

1. Рядок: "It has become appallingly obvious that our technology has exceeded our humanity. - Albert Einstein". Виведіть рядок у зворотному порядку.
2. Рядок: "Innovation distinguishes between a leader and a follower. - Steve Jobs". Виведіть всі букви рядка, які стоять на непарних позиціях.

Варіант 24:

1. Рядок: "The machine does not isolate man from the great problems of nature but plunges him more deeply into them. - Antoine de Saint-Exupery". Знайдіть всі слова, що закінчуються на "e".
2. Рядок: "Technological progress has merely provided us with more efficient means for going backwards. - Aldous Huxley". Виведіть усі слова у алфавітному порядку.

Варіант 25:

1. Рядок: "We are becoming the servants in thought, as in action, of the machine we have created to serve us. - John Kenneth Galbraith". Перевірте, чи міститься слово "technology" у рядку.
2. Рядок: "Technology made large populations possible; large populations now make technology indispensable. - Joseph Wood Krutch". Замініть всі входження слова "technology" на "TECHNOLOGY".

Контрольні запитання

1. Що таке регулярний вираз (regex) і як його можна використовувати в Python?
2. Які основні функції надає модуль re Python для роботи з регулярними виразами?

3. Як використовувати `re.findall()` для пошуку всіх збігів регулярного виразу в рядку?
4. Як використовувати `re.split()` для розбиття рядка за допомогою регулярного виразу?
5. Як використовувати `re.sub()` для заміни збігів регулярного виразу в рядку?
6. Які спеціальні символи використовуються в регулярних виразах і що вони означають?
7. Чому важливо використовувати `r` перед рядком регулярного виразу в Python?

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Кренивч А. П. Python у прикладах і задачах. Частина 1. Структурне програмування. Навчальний посібник із дисципліни "Інформатика та програмування" – К.: ВПЦ "Київський Університет", 2017. – 206 с.
2. Костюченко А. О. Основи програмування мовою Python: навчальний посібник. Ч.: ФОП Баликіна С.М., 2020. 180 с.
3. Яковенко А. В. Основи програмування. Python. Частина 1 [Електронний ресурс] : підручник для студентів які навчаються за спеціальністю 122 «Комп'ютерні науки» спеціалізацією «Інформаційні технології в біології та медицині» / А. В. Яковенко ; КПІ ім. Ігоря Сікорського. – Електронні текстові дані (1файл: 1,71 Мбайт). – Київ : КПІ ім. Ігоря Сікорського, 2018. – 195 с.
4. Mark Lutz, Learning Python, 4th Edition. – Sebastopol, CA: O'Reilly Media, Inc., 2009. – 1161 p.
5. Дунець Р. Б. Арифметичні основи комп'ютерної техніки / Р. Б. Дунець, О. Т. Кудрявцев. – Львів : Ліга-Прес, 2006. – 142 с.
6. Довгалець С. М. Алгоритмічні мови та програмування. Частина 1. Основи інформатики та комп'ютерної техніки. Навчальний посібник / С. М. Довгалець, Р. В. Маслій. – Вінниця : ВНТУ, 2009. – 116 с.
7. Яковенко А. Основи програмування: методичні вказівки до виконання комп'ютерних практикумів з дисципліни "Основи програмування". Основи програмування мовою Python / А. В. Яковенко. – Київ : НТУУ "КПІ ім. І. Сікорського", 2017. – 87 с.
8. Електронний путівник мовою програмування Python. [Режим доступу]: <https://pythonguide.rozh2sch.org.ua/>
9. Онлайн платформа та БД для вивчення алгоритмів та мов програмування. [Режим доступу] <https://leetcode.com/>